T-SQL

- SELECT statement
- The SELECT statement has 6 principal clauses:

SELECT clause

- This is where you say what columns or calculated values to show.
- You can rename (alias) columns using OriginalName AS NewName.
    - The word "AS" is optional, but is recommended.
- If your names have a space or other special character, then they would need to be enclosed in [ ] hard brackets.
    - The use of special words, like Name, may be possible, but is not recommended.
- You can use DISTINCT to remove duplicates, and TOP number or TOP (number) to show first few rows.
    - TOP is generally used in combination with ORDER BY.
- You can use aggregations - for example, SUM, COUNT, MIN, MAX and AVG
    - COUNT(*) counts all the rows. COUNT(ColumnName) only counts those rows where ColumnName IS NOT NULL.
    - AVG is the SUM divided by the COUNT of the NOT NULL rows.
    - SUM and AVG can only be used for numerical data.
    - If you have a SELECT clause which includes aggregations and non-aggregated columns, you MUST use a GROUP BY clause.

FROM clause

- This shows the source data, either a table, view, or query.
    - A query used as a source is called a "derived query".
    - If you do this, you WILL need to alias it.
- You can alias the source tables as well.
    - This is more often used when you have more than one source.
- If using more than one table, you will need to JOIN them together.

- All joins apart from the CROSS JOIN need an ON. This condition shows how the two sources are related.
- The JOINs are:
    - INNER JOIN or JOIN. Where both rows match.
    - LEFT [OUTER] JOIN. All the rows from the first-mentioned source (before the word JOIN) and only those in the second source which match,
    - RIGHT [OUTER] JOIN. All the rows from the second source and on those in the first which match,
    - FULL [OUTER] JOIN. All the rows in both sources, after seeing what matched.
    - CROSS JOIN. Each row in the first source matches each row in the second source. Not often used.

WHERE clause

- This filters the rows based at least one condition.
- The standard comparators are:
    - = - to be the same as or equal to,
    - > - greater than,
    - >= - greater than or equal to,
    - < - less than,
    - <- - less than or equal to, and
    - <> or != - not equal to.
- Text and date literals are enclosed in apostrophes.
    - For NCHAR and NVARCHAR text literals, you would add a capital N before the first apostrophe.
    - Date literals are generally represented as:
        - 'yyyy-mm-dd', or if you have time as well
        - 'yyyy-mm-dd hh:mm:ss.fffffffff' or 'yyyy-mm-ddThh:mm:ss.fffffffff'.
            - You should use 2 digits for hours, minutes and seconds.

        - f stands for fractional seconds, and you can use between 1 and 9 decimal places, depending on the data type.
    - Text comparisons can also use LIKE, which can compare with the start, middle and/or end of strings, using:
        - The two wildcards:
            - % - this stands for 0, 1, 2 or more characters, and
            - _ - this stands for exact 1 character.
        - [BM] to search for either the characters B or M,
        - [B-M] to search for the characters from B to M, and
        - [^B-M] to search for any characters apart from those between B and M.
    - You can also use NOT LIKE.
    - Text comparison can also use COLLATE to change the collation (how text is sorted), which includes:
        - CS or CI - case-[in]sensitive. Whether 'b' is the same as 'B', and
        - AS or AI - accent-[in]sensitive. Whether 'í' is the same as 'i'.
- You can also search for multiple conditions using:
    - AND - both conditions on either side of the AND must be true for the entire condition to be true,
    - OR - either condition must be true.
    - You can also use NOT to change true to false, and false to true.
    - You can also use brackets/paratheses to group conditions together, or to make it more readable.
- You can also use BETWEEN to search for an inclusive range (ColumnName BETWEEN FirstValue AND Second).
- To search for missing data, you can use IS NULL (one word).
    - You can also use IS NOT NULL.
- You cannot use any aliases declared in the SELECT clause.

GROUP BY clause

- If you use aggregations only in the SELECT clause, you do not use a GROUP BY clause.
- If you use only non-aggregations in the SELECT clause, you need not use a GROUP BY clause.
    - If you do so, it acts like DISTINCT.
- If you use both aggregations and non-aggregations in the SELECT clause, you MUST use a GROUP BY clause.
    - You must put in the GROUP BY clause all of the non-aggregated columns which are shown in the SELECT clause.
- You cannot use any aliases declared in the SELECT clause.

HAVING clause

- This can be used in conjunction with the GROUP BY clause.
- If you want to filter the rows after the GROUP BY clause, then you can do so in the HAVING clause.
- This is generally used with conditions which involve an aggregation.
- You cannot use any aliases declared in the SELECT clause.

ORDER BY clause

- This orders the result, either ASCending or DESCending.
    - You use ASC or DESC, not ASCENDING or DESCENDING.
    - The default is ASC.
- You should specify the column name, and you use either the original name or the alias.
    - You could also use the column position - for example, 2 - but that is not recommended.
- If there are multiple rows tied after ordering, the final position is left to SQL Server.
    - This is called a non-deterministic order.
- The use of ASC or DESC is limited to just the column that it precedes.
    - You can use either multiple times in an ORDER BY clause, but just once per column sorted.

- NULLs are treated as being smaller than the smallest value.
    - So in ASCending order, they are first, and in DESCending order, they are last.

Design and implement database objects

1a-c. Design and implement tables, including data types, size, columns

Exact number data types

- The exact number Data types which do not allow for decimal numbers are:
    - bit - from 0 to 1. Up to 8 bits can be stored as 1 byte of storage in total.
        - You can use COUNT for bit columns, but not SUM, AVG, MIN or MAX.
    - tinyint - from 0 to 255. Each tinyint requires 1 byte of storage.
    - smallint - from -32,768 to 32,767. It requires 2 bytes.
    - int - from -2,147,483,648 to 2,147,483,647. It requires 4 bytes.
    - bigint - from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- Number literals expressed without decimal places are converted into tinyint, smallint, int or decimal (not bigint).
    - You can use +, -, *, / and % (modulo - the remainder)
    - An integer divided by an integer results in an integer, truncating any decimal places in the result.
- The exact number Data types which do allow for decimal numbers are:
    - decimal and numeric. These synonyms can use 2 arguments:
        - Precision - the number of decimal digits to be stored, both before and after the decimal point, from 1 to 38.
            - The default is 18.
            - It needs 5 storage bytes, plus 4 additional bytes when precision is at or over 10, 20 or 29.
        - Scale - the number of decimal places needed.
            - This can only be up to the Precision.

    - The default is zero.

- smallmoney and money. These are int and bigint divided by 10,000.
    - Smallmoney can store between -214,748.3648 to 214,748.3647, and requires 4 bytes.
    - Money can store between -922,337,203,685,477.5808 to 922,337,203,685,477.5807, and requires 8 bytes.
    - Currency information is not stored.

Floating number data types

- Approximate numerics are stored in the float and real data types. The two possibilities are:
    - float(24), also known as real. This stores precisely up to 7 digits and takes 4 bytes.
    - float(53). This stores precisely up to 15 digits and takes 8 bytes.
- After the precise number of digits, other digits are stored approximately.
    - float(53) can use numbers with up to 308 digits to the left or right of the decimal point.
    - float(24) or real can use numbers with up to 38 digits to the left or right of the decimal point.
- Why use float instead of decimal/numeric?
    - When you are using numbers where a close approximation is acceptable, and where number stored are very large or very small.
    - When storage costs needs to be reduced.
        - float(24)/real takes 4 bytes, but decimal starts at 5 bytes.
    - It is also used in the Vector data type when using AI Embeddings (see topics 53-73).
    - You can also use float16 when using the Vector data type, which only uses 2 bytes per value.
- Because it is approximate, exact comparisons are not really possible.
    - Do not use = or <> operators; only use > and <.

String data types

- Characters can be stored using:

- char - this stores an exact length of string.
    - The number of bytes stored is given as an argument, between 1 and 8,000. The default is 1.
        - In Latin encoding character sets, one character takes one byte. However, in other encoding sets, characters may take two or more bytes.
    - The storage required is the number of bytes stored.
    - Only use when you have strings which are consistently the required length (or are 1 or 2 less).
- varchar - this stores a variable-sized length of string.
    - The maximum number of bytes that can be stored is from 1 to 8,000. There is no default.
    - Alternatively, you can use varchar(max) to store up to 2Gb (1 Mb in Fabric Data Warehouse).
    - The storage requires is the actual length of the string (in bytes) plus 2 bytes.

- To store text using the UTF-16 character encoding, use nchar and nvarchar.
    - Storage costs are double for nchar, and nearly double for nvarchar.
        - It is twice the byte length for nchar.
        - It is the actual number of bytes stores plus 2 for nvarchar.
        - Note: in some encoding, characters may take more than 2 characters.
- To store large text, you can use text and ntext data types.
    - However, these will be removed in a future version of SQL Server. You should use varchar(max) and nvarchar(max) instead.
    - These can store up to 2 Gb.

Date/time data types

- Dates and times can be stored in the following data types:
    - date
        - These store dates from 1 January 0001 to 31 December 9999.
        - No times are stored.

- Storage size is 3 bytes.

- time
  - These store times from midnight to just before midnight, but not dates.
  - It does not store time zone data.
  - For the seconds, it can store from 0 to 7 decimal places. The default is time(7).
    - time(0) to time(2) requires 3 bytes.
    - time(3) and time(4) requires 4 bytes.
    - time(5) to time(7) requires 5 bytes.
- smalldatetime
  - This stores date and time from 1 January 1900 to 6 June 2079.
  - It stores to the nearest minute.
  - It requires 4 bytes.
- datetime
  - This stores date and time, up to 1/300th of a second.
  - The fractional seconds are either '.xx0', '.xx3' or '.xx7'.
  - It requires 8 bytes.
- datetime2 and datetime2(n)
  - This stores date and time, combining the date and time data types.
  - As such, it requires 6-8 bytes.
  - The default is datetime2(7), which requires 8 bytes.
- datetimeoffset
  - This is as datetime2, but also stores the time zone information, from -14:00 to +14:00.
  - You cannot use a date literal with a T to separate the date and time - it needs a space.
  - You also need a space before the time zone offset.
  - It requires 10 bytes.

Other data types

- Other data types include:
    - binary and varbinary
        - This is used to store binary (non-textual or compressed) data, such as images, audio, executable files, and logical flags (true/false).
        - As per char and varchar, it can store 1 to 8000 bytes, or use (max) to store up to 2Gb.
    - cursor - these are used to go through datasets, one row at a time.
    - geography - this spatial data type represents location data on a sphere, including latitude/longitude coordinates.
    - geometry - this spatial data type represents location data on a flat plane.
    - hierarchyid - represents a position in a tree hierarchy (like employees in a company)
    - json - documents in a JSON format (see topics 3 and 14),
    - vector - this stores vector data for machine learning applications and similarity search, as used with AI (see topics 53 onwards),
    - rowversion - this is a binary number which is incremented when an data is updated in a table using insert or update,
    - sql_variant - this can store a variety of data types,
    - table - this stores a result set temporarily.
    - uniqueidentifier - this stores a 16-byte GUID (Globally Unique IDentification Number)
    - xml - this stores XML data.

1d. Design and implement tables, including indexes

- You can create an index, which may improve performance for SELECT statements, specially where they use relevant columns in:
    - a JOIN in the FROM clause,
    - a WHERE or GROUP BY clause,
    - an ORDER BY clause (if the index matches the sort order),
    - a MIN or MAX aggregation query.

- It can also speed up queries where all of the columns are in the index.
- However, it would slow down INSERT, UPDATE and DELETE operations, because the index would need to be updated.
- WHERE clauses which are able to be sped up by indexes are called SARGable queries. (SARG stands for Search ARGument.)
- SARGable queries include WHERE ColumnName:
    - BETWEEN
    - =, <, <=, >, >=
    - IS NULL or IS NOT NULL,
    - IN
    - LIKE 'Term%'
- Queries which are generally not improved include:
    - <>
    - NOT, NOT IN, NOT LIKE
- Non-SARGable queries include:
    - Using most functions. For example:
        - WHERE Column + 1 = 2
        - YEAR(DateColumn). Instead, use >='FirstDayOfYear' AND < 'FirstDayOfNextYear'.
    - LIKE '%Term' or '%Term%'. Because the search term is in the middle/end of a string, it cannot use an alphabetical index to find these terms.
- When you create a PRIMARY KEY or UNIQUE constraint, an index is automatically created.
    - A Primary Key creates by default a CLUSTERED index, which sorts the entire table in its order.
    - A UNIQUE Key creates by default a NONCLUSTERED index, which creates an index which is separate from the table.
- To create an index, use:
    - CREATE
    - Optionally, UNIQUE

- Optionally, CLUSTERED or NONCLUSTERED
- INDEX NameOfIndex ON ObjectName
- (ColumnName and optionally ASC DESC. Multiple ColumnNames are separated by a comma)
    - You can have a maximum of 32 key columns and a key size of 1,700 bytes.
    - The order of the columns can affect the performance of the index.
- Optionally, INCLUDE (ColumnNames)
    - Include columns are non-key columns.
    - The order of non-key columns do not by itself affect the performance of a query.
    - You can use all data types apart from text, ntext and image columns.
    - Included columns cannot be dropped or changed from a table unless the index is first dropped, except:
        - changing NOT NULL to NULL (or the other way round), or
        - increasing the length of varchar, nvarchar, and varbinary columns.
    - Should a column be a key column, included, or not used in the index?
        - If they are used in the JOIN, WHERE, GROUP BY or ORDER BY, they probably are key columns.
        - If they are used in the SELECT clause, if they are used as an INCLUDEd column, then the index would covered - the query would just use the index, and not a JOIN between the index and the table.
- Optionally, WHERE condition - this creates a filtered index.
    - Do this if you only need to speed up SELECT statements which use this WHERE condition.
- Optionally, options which are added in a WITH ( ) include:

    - PAD_INDEX = ON (or OFF) and FILLFACTOR = number between 0 and 100. The FILLFACTOR shows how fill the individual index pages will be when it is created or rebuilt.
        - The smaller the number, the more pages will be used (and so the larger the index), but the more space there is for insertions into the index.
    - DROP_EXISTING = ON, which rebuilds the index with modified column specifications but using the same index name.

- If the index is too fragmented (data in different places in the database), you can also rebuild the index using:
    - ALTER INDEX IndexName ON Object REBUILD
    - You can also add the options within WITH ( ).
    - Rebuild is by default offline (the index is not usable). You can REBUILD WITH (ONLINE = ON).
    - For rowstore indexes (as opposed to column store indexes), you could use REORGANIZE instead of REBUILD.
        - This is less resource intensive.
        - This is always performed online.
- When indexes are created, in addition to the column(s) which are mentioned in the individual, also:
    - for tables with a clustered index, the clustered index key is added, or
    - for tables without a clustered index, the Row Identifier is added to locate the row.
- You can temporarily turn off an index by using
    - ALTER INDEX IndexName ON Object DISABLE.
    - You can ENABLE it later.
- You can also delete it by using:
    - DROP INDEX IndexName ON Object

1e. Design and implement tables: column store indexes

- The traditional index is called a "rowstore index".

    - All of the columns of a row are stored, then the next row, then the next row.

- An alternative is the columnstore index.
    - This was first introduced in SQL Server 2012, but has been developed since then. These notes concentrate on the SQL Server 2016 version, with improvements in the SQL Server 2025 version.
    - For Azure SQL Database, it is not available for the Basic tier, or for the Standard tier below S3.
- To create a columnstore index, add the word "COLUMNSTORE" before the word "INDEX".
    - You cannot use INCLUDE keyword.
- A columnstore index creates indexes for individual columns.
    - It is stored in column segments, which contains one column for an individual rowgroup (usually 1,048,576 rows).
    - Unlike rowstore indexes, the data is not necessarily ordered.
    - Because it contains similar data, this data can be compressed, often by 10 times, to reduce storage costs and query execution time.
- Like rowstore indexes, you can create clustered and nonclustered indexes.
    - A clustered columnstore index is the table's main storage.
    - A nonclustered columnstore index is a separate index, which can be a filtered index.
    - You can also create ordered nonclustered columnstore indexes.
        - It first became available in SQL Server 2025, together with Azure SQL Database, Managed Instance, and SQL Database in Fabric.
        - This can make queries with a WHERE clause much faster.
        - However, it takes longer to load data into an ordered columnstore index.
        - To create an ordered index, add ORDER (column names) - note, it is ORDER, not ORDER BY. You cannot use ASC or DESC.
- You should consider using columnstore indexes for queries which scan the entire table:
    - Clustered, to store:

1e. Design and implement tables: column store indexes

- data warehousing tables (fact tables and large dimension tables), or
- Internet of Things workloads which insert data, but which have limited updates and deletes.

- Nonclustered, potentially ordered, to do real-time analysis.
  - This might replace an existing rowstore index.
  - It might also replace a data warehouse, so you query using the columnstore indexes.
  - You can create a secondary nonclustered index on a clustered table, for example as primary or foreign key constraints.
- Ordered, when:
  - You are doing real-time analysis. For example, when you are querying the last 30 seconds of data using an ordered columnstore index based on the date of insertion.
  - The data is not often updated or deleted,
  - The data in the first column of the index key is more distinct (not repeated), including GUIDs (Globally Unique Identifers).
  - The index key is often queried, especially its first column.

- Note - adding data takes longer in an ordered columnstore index because it needs to sort the compressed data.
- Do not use columnstore index when:
  - You need varchar(max), nvarchar(max) or varbinary(max) data types in the index.
  - The data is temporary,
  - The data has less than 1,000,000 rows per partition,
  - The data is not stable, with more than 10% of DML operations being updates and deletes, as this will cause fragmentation, and you would need to reorganize. This is especially the case for ordered indexes.
- You should use rowstore indexes when your queries look for a particular value or a small range of values (using a seek).
- You can have a rowstore table and a nonclustered columnstore index on the same table.

- You can also add partitioning to columnstore indexes.
    - Your queries might then only query certain partitions.
    - It is best that each partition has at least 1,000,000 rows.
    - You can then compress less-used partitions using the WITH (DATA_COMPRESSION = COLUMNSTORE_ARCHIVE) compression option.
    - This decreases storage by around 30%, but also decreases query performance on that partition.
- Regarding the ALTER INDEX statement:
    - You can reorganize, disable and rebuild indexes.
    - Apart from that, you cannot use ALTER INDEX with columnstore indexes - you must drop and recreate the index instead.

2a. Design and implement specialized tables: in-memory

- Memory-optimized tables, also known as in-memory tables, are stored in the main memory.
    - This improves performance, as data access and transaction execution is more efficient.
        - One company improved performance by up to 30 times. Another by 2 times.
        - The best performance is achieved using natively compiled stored procedures.
        - These are compiled at create time, instead of during the first execution.
        - They cannot use the TOP expression if they return more than 8,000 rows.
        - To create a natively compiled stored procedure, use CREATE PROCEDURE ... WITH NATIVE_COMPILATION.
- For Azure SQL Database, memory-optimized tables:
    - are available in the Premium and Business Critical service tiers,
    - some functionality is available in the Hyperscale service tier.
- By default, all transactions written to memory are simultaneously written to disk.
    - If the server restarts, the memory-optimized version is restored from disk.

- To create an in-memory table, add at the end of it:
    - WITH (MEMORY_OPTIMIZED=ON)
    - The default value is (MEMORY_OPTIMIZED=ON, DURABILITY = SCHEMA_AND_DATA)
- If you want greater performance, you can use memory-optimized tables with transaction durability delayed.
    - Transactions which have been committed to the in-memory table are soon after committed to the disk version.
    - This means that, if there is a crash or failover, some transactions may be lost.
    - To do this run:
        - ALTER DATABASE DatabaseName
        - SET DELAYED_DURABILITY = ALLOWED
    - The opposite is = DISABLED.
- You can also have non-durable memory-optimized tables.
    - Transactions are not logged or persisted on disk.
    - If there is a crash or failover, the data will be lost.
    - To create such a table, use:
        - (MEMORY_OPTIMIZED=ON, DURABILITY = SCHEMA_ONLY)
- Indexes are not logged on disk.
    - You cannot use clustered indexes. Primary keys would need to be defined with non-clustered indexes (PRIMARY KEY NONCLUSTERED).
    - Indexes are created using either:
        - Hash index. This is an array of pointers.
            - Excellent performance when WHERE uses an exact value (point lookup) for each column in the hash key,
            - Poor performance when WHERE uses a range of values.
            - Poor performance when values are not specified for each column in the key.
            - Fine for retrieving all table rows.

        - To create one, use ALTER TABLE NameOfTable ADD INDEX NameOfIndex [UNIQUE] HASH (ColumnNames) WITH (BUCKET_COUNT = 64)
        - The bucket count should 1-2 times the number of distinct values in the index key.
            - Performance is still good with 10 times of the actual number of key values.
            - Overestimating is generally better than underestimating.
    - Memory-optimized nonclustered index. Best for:
        - ORDER BY clauses on the indexed column(s), in the order of the index (ASC or DESC)
        - Queries where only the first column(s) of the index are tested.
        - WHERE queries which are not = (so <>, or >=).
        - Fine for retrieving all table rows.
        - You create it as per a normal nonclustered index.

- Rows in memory-optimized tables are versioned.
- Memory-optimized nonclustered indexes have better performance than disk-based versions.
    - For point lookups (using an = sign in the WHERE), memory-optimized hash indexes have even better performance.
- The disadvantages of in-memory tables are:
    - it uses memory which is not available to other objects, which can reduce their performance,
    - you cannot use clustered indexes.
- You should consider using in-memory tables:
    - When you have large volumes of transactions, and you need low latency for individual transactions.
        - This can include financial trading, sports betting, mobile gaming, and advert delivery.

- When you ingest large amounts of data from multiple sources at the same time.
    - This can include inserting sensor readings/events, and updating data from multiple sources.
- When you need to Maintaining session state and caching data (for example, in an ASP.NET application)
- For temporary tables, table variables, and table-valued parameters.
- When loading data into a staging table, transforming the data, and loading into final tables (ETL - extract, transform, load)

2b. Design and implement specialized tables, including in-memory, temporal, external, ledger, and graph

- A temporal table is one that allows for retrieval of data at any point in time.
    - You can audit all data changes and allowing data forensics,
    - Get the table at any time in the past.
    - Use this to create trends over time.
    - Allows for SCD (slowly changing dimensions) for decision support applications,
    - Restore data if there is a bad data change or app error.
- The temporal table:
    - two columns with datetime2 data type: ValidFrom and ValueTo columns
        - They record the start and end date/time for that row.
        - These contain the “current value” for the row.
    - a reference to a history table with a mirrored schema.
        - It records each previous version of each row.
        - You can specify an existing history table or create a default history table.
- To create a temporal table, you need:
    - CREATE TABLE dbo.Employee (...
    - [ValidFrom] DATETIME2 GENERATED ALWAYS AS ROW START,
    - [ValidTo] DATETIME2 GENERATED ALWAYS AS ROW END,

- PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo))
- WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.EmployeeHistory));

2c. Design and implement specialized tables: external

- You can create tables, where the schema (column names, data types and so on) are in SQL Server, but the data is outside of SQL Server.
- For Azure SQL Database and SQL Database in Fabric, it can be used for data virtualization using PolyBase.
    - You can use T-SQL to query data in SQL Server, Oracle, Teradata, MongoDB, Hadoop clusters, Cosmos DB, and S3-compatible object storage.
    - For additional sources, you can also use third-party ODBC drivers.
    - It allows querying from:
        - Relational tables,
        - Semi-structured data, and
        - structured file-based data format.
        - For example, CSV, Parquet files, files in JSON format, and Delta Lake files.
    - This allows for:
        - Using external data as if it was in SQL Server,
        - Moving older, less-used data, onto other platforms,
        - Reducing complexity of integrating external data source.
        - Allowing for real-time insights,
        - Using SQL Server security features.
    - To use it, you would need:
        - A Master Key Encryption,
        - A Database Scoped Credential using either a Shared Access Signature (SAS) and Secret, or a Managed Identity, to allow authentication and authorization to the data source.
        - An External Data Source which uses the Database Scoped Credential,

    - An External File Format to specify what format the data is in, and
    - CREATE EXTERNAL TABLE NameOfTime (structure)
    - WITH (DATA_SOURCE = ExternalDataSource,
    - LOCATION = 'Location',
    - FILE_FORMAT = ExternalFileFormat);

- For Azure SQL Database, it can also be used for elastic queries.
    - You can then query multiple databases, including remote Microsoft and non-Microsoft tools, to get a single result.
        - For example, Excel, Power BI and Tableau.
        - It cannot be used against data sources which are not databases.
    - You can employ:
        - Vertical partitioning (cross-database querying) - different tables are on different databases, or
        - Horizontal partitioning (sharding) - tables with the same schema on different databases have got different parts of data.
    - Again, you would need to create a:
        - Master Key,
        - Database Scoped Credential,
        - For horizontal partitioning, you also need to create a shard map, which maintains the mapping of the databases (shards),
        - External Data Source - it would be of type:
            - RDBMS for vertical partitioning, or
            - SHARD_MAP_MANAGER for horizontal partitioning (as of April 2027, this will be discontinued, and you would need to use more specific options for the relevant data source), and
        - External Table.
    - You can then access the External Table as if it were a single table.

2d. Design and implement specialized tables: ledger

- Ledger tables provide tamper-evidence capabilities for your data.
    - It preserves historical data.

    - It allows you to attest to auditors and others that your data has not been tampered with.
    - It protects data from attackers or high-privileged users.
- Rows modified in a ledger table are cryptographically hashed (like creating a fingerprint).
    - That, along with other hashes, creates a blockchain.
    - The latest block is called the database digest - this is the then state of all database ledger tables.
- They can be stored outside the database in tamper-proof storage - for example:
    - Azure Blob Storage configured with immutability policies,
    - Azure Confidential Ledger or
    - on-premises Write Once Read Many (WORM) storage devices.
- There are two types of ledger tables:
    - Append-only ledger tables - this only allows insertions
        - This is useful for SIEM (security information and event management) applications
    - Updatable ledger tables, which also allow updates and deletions.
        - This is useful for System of Record (SOR) applications.
        - These require a history table, which is automatically created when you create the updatable ledger table.
- You can also create a ledger database, which only contains ledger tables (not regular tables).
    - In Azure SQL Database, when you create an Azure SQL Database, this is available in the Security tab - Ledger, and check "Enable for all future tables in this database".
    - The default for all tables in a ledger database is Updatable ledger tables.
- To create an append-only ledger table, add at the end of the table
    - WITH (LEDGER = ON (APPEND_ONLY = ON))
- To create an updatable ledger table, add at the end of the table
    - WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = HistoryTable), LEDGER = ON);

- You can then use the ledger_start_transaction_id and ledger_start_sequence_number (and in the history table, ledger_end), which shows what data was inserted/changed at the same time.
    - You can use this for the ledger table and, for updatable ledger tables, the history table.
- To verify the digest storage, you can use EXEC sp_generate_database_ledger_digest

2e. Design and implement specialized tables: graph

- To create a node table, add AS NODE at the end.
    - This creates the table with a column starting with $node_id to identify each node.
- To create an edge table, add AS EDGE at the end.
    - This creates the table with the following additional columns:
        - $edge_id,
        - $from_id and $to_id to go between the nodes.
- Continued in topic 17.
- 6. Design and implement partitioning for tables and indexes
- If you have a big table, you might want to divide it into smaller parts of a table called partitions.
    - These partitions can be stored in the same file or filegroup (for example, the PRIMARY group)
    - For on-premises SQL Server or Azure SQL Managed Instance, you can use different files (or even different drives), which can speed up retrieval.
- The advantages include:
    - Using partitions can speed up queries which target data in one (or a few) partitions.
    - You can also performance maintenance on just one (or more) partitions at a time, instead of the entire table.
    - Reading or updating data can lock other people's access to individual rows, pages (group of rows), or even the entire table. Adding partitions can mean that, instead of the entire table being locked, just a partition could be locked.

    - This can increase the number of people who can access the entire table. Also, see topic 34 for other techniques.

- You can switch partitioned data into an archive table.

- The disadvantages include:
    - More partitions will use more memory.
    - If you run a query using TOP, MAX or MIN, or without a WHERE, then performance may be decreased, as all partitions need to be evaluated.
- If you run queries against two partitioned tables, then the partitioning should be done in the same way for best performance.
- You can also create indexes which uses the same partitioning as the table - these are called aligned indexes.

3. Introducing the JSON structure and data type

- JSON is a text-based data format used for communicating with web and mobile apps:
    - Log files,
    - NoSQL Databases, and
    - REST (Representational State Transfer) web services, including Microsoft Azure services.
- JSON is available in SQL Server 2016 or above.
- You can store JSON data in a json data type.
    - It was first introduced in SQL Server 2016.
- It is more effective that storing in a [n]varchar format, as it allows:
    - Swifter reads, because the data is parsed,
    - Swifter writes, because the functions can update an individual value within the document,
    - More compressed file size.
- When using JSON, you can:
    - Simplify data models by using JSON instead of tables connected to more tables,
    - Store data with variable attributes, as the JSON format is flexible,

    - Analyse data using native T-SQL functions,
    - Load the data into SQL Server, without it needed to be staged and converted,
    - Use it with REST API (Application Programming Interfaces)
- JSON is written:
    - in key/value pairs – for example: {“name”: “Phillip”}
    - Multiple key/value pairs can be enclosed in the same { }
        - For example: {“Given name”: “Phillip”, “Family name”: “Burton”}
    - Arrays (where there *may* be more than one object) is written in [ ]
        - For example: [“Given name”: “Phillip”, “Family name”: “Burton”} , “Given name”: “Jane”, “Family name”: “Smith”}]
- When using the JSON data type, it will check that the data is valid before it is stored.

3. Design and implement JSON columns and indexes

- To add a column using the json data type, you can use a CREATE TABLE or ALTER TABLE command.
    - You can also specify NULL or NOT NULL.
    - You can also use CHECK and DEFAULT constraints.
- To speed up retrieval from a json column, you can create an index.
    - This requires SQL Server 2025 or later.
    - It can be created with or without the data being in the table,
    - The table needs a clustered primary key beforehand.
    - You can only have one JSON Index for a particular column.
    - It cannot be used for views, table-valued variables, or memory optimized table, or for computed json columns.
- You create the index using:
    - CREATE JSON INDEX IndexName ON TableName (ColumnName)

4a. The PRIMARY KEY Constraint

- A PRIMARY KEY is a column or columns which uniquely define each row.

    - If you use more than one column, it is called a compound key.
    - There cannot be a duplicate of the value in this column, or the combination of values in these columns, in the data.
    - Generally, the shorter the data types of the PRIMARY KEY, the better, as the values of the PRIMARY KEY will be used in other indexes and foreign constraints.
- You can only have one PRIMARY KEY per table.
    - It needs to be based on NOT NULL column(s), unless you are using memory-optimized tables.
- Creating a PRIMARY KEY will create an index on the table.
    - If you do not specify otherwise, it will be a CLUSTERED index. This means that the data will be sorted based on the PRIMARY KEY.
- To create a PRIMARY KEY, either:
    - Create it in-line with the relevant column (single):
        - CREATE TABLE TableName
        - (ColumnName DataType NOT NULL PRIMARY KEY)
        - You can only do this for a single column.
        - You cannot name the PRIMARY KEY - it will be given an automatic name
    - Create it separately as part of the CREATE TABLE:
        - CREATE TABLE TableName
        - (ColumnName DataType NOT NULL,
        - CONSTRAINT PKName PRIMARY KEY(ColumnName))
    - Create it separately to the CREATE TABLE:
        - ALTER TABLE TableName
        - ADD CONSTRAINT PKName PRIMARY KEY(ColumnName)
- To remove a PRIMARY KEY, use
        - ALTER TABLE TableName
        - DROP CONSTRAINT PKName
- To modify a PRIMARY KEY, DROP it and then ADD CONSTRAINT.

- A non-compound Primary Key is often used with DEFAULT constraints.

4b. The UNIQUE Constraint

- UNIQUE constraints show a column or columns which are not to be duplicated.
- When you create a UNIQUE constraint, a UNIQUE key is created.
    - This can speed up queries.
- The differences between UNIQUE and PRIMARY KEY include:
    - You can only have one PRIMARY KEY in a table. You can have multiple UNIQUE constraints in a table.
    - PRIMARY KEY constraints do not allow for NULLs. UNIQUE can allow for a maximum of one NULL in a column.
    - Indexes
        - A PRIMARY KEY, by default, creates a Clustered Key, which orders the table based on the PRIMARY KEY.
        - A UNIQUE constraint, by default, creates a non-clustered index, which does not sort the table, and creates the index separately from the table.
    - You can create and drop UNIQUE CONSTRAINTs in the same way as in PRIMARY KEYs.

4c. The FOREIGN KEY constraint

- FOREIGN KEYs restrain the data that is in one table, based on the data in another table.
    - For example, you might want to stop test results from being entered for pupils which are not in a pupil table.
    - This is called enforcing referential integrity.
- The Column name(s) need to have a PRIMARY KEY or UNIQUE constraint.
- To create a FOREIGN KEY relationship, use either:
    - In the table definition, use:

        CONSTRAINT ConstraintName

        FOREIGN KEY (ColumnNames)

        REFERENCES OtherTable(ColumnNames)

- Add a CONSTRAINT, using:

  ALTER TABLE TableName

  ADD CONSTRAINT PKName ConstraintName

  FOREIGN KEY (ColumnNames)

  REFERENCES OtherTable(ColumnNames)

- If you have existing data that would violate the FOREIGN KEY, it will not be created, unless you use:
  - ALTER TABLE TableName WITH NOCHECK

- You can DROP the constraint.
  - You can also disable it using:
    - ALTER TABLE TableName
    - NOCHECK CONSTRAINT ConstraintName
  - To reenable it, you can substitute NOCHECK with:
    - WITH NOCHECK CHECK - do not validate existing data, or
    - WITH CHECK CHECK - validate existing data.
  - To modify a constraint, you need to DROP and then CREATE it again.
- DELETE statements will fail if a FOREIGN KEY constraint stops it. However, you can add ON UPDATE / ON DELETE and then:
  - NO ACTION - this is the default, and raises an error.
  - CASCADE - any changes to the parent table are reflected in the referencing table
  - SET NULL - any changes to the parent table means that the reference is set to NULL
  - SET DEFAULT - any changes to the parent table means that the reference is set to DEFAULT.
    - This requires a DEFAULT constraint to have been created. If you do not have one, it sets it to NULL.

4d. The CHECK Constraint

- You can limit the data that can be entered using a CHECK constraint.

    - Using NOT NULL as part of the table definition does act like a CHECK constraint.
- You put the valid conditions after the word CHECK in brackets/parentheses.
    - Invalid data is when the condition evaluates as FALSE.
    - If the data is NULL, then the condition might evaluate as unknown instead of NULL. It is best to evaluate NULL separately.
    - You can compare one column against another using a CHECK constraint that has been create not in-line.
- You can create and drop CHECK CONSTRAINTs in the same way as in PRIMARY KEYs.

4e. The DEFAULT Constraint

- You can add a DEFAULT value to a column.
    - Each DEFAULT constraint only operates on one column, not multiple.
    - Only one DEFAULT value can be used per column.
- This means that, if you add a row and there is a NOT NULL column, the row can still be inserted using the DEFAULT value.
- You will often use DEFAULT using a PRIMARY KEY or a SEQUENCE.
- You create an inline DEFAULT constraint using DEFAULT DefaultValue as the end of the column definition.
- You can also create it as a standalone constraint using:
    - CONSTRAINT ConstraintName DEFAULT DefaultValue FOR ColumnName.
- You cannot use create it at the end of a table definition.
- DEFAULT values are often used for:
    - Primary Keys,
    - CreatedDates, using the DEFAULT getdate() or sysdatetime(), for example.
    - IDENTITY(StartNumber, Increment) for auto-incrementing numbers - You do not use the word "DEFAULT" for IDENTITY.
        - None of the arguments are required. The default is StartNumber 1 and Increment 1.

    - If you are using a memory-optimized table, then you must use the 1 and 1 values.
    - You cannot override the number used in an IDENTITY column, unless you set IDENTITY_INSERT to OFF.
    - Only one IDENTITY column can be used in a table.
    - You cannot remove the IDENTITY property without DROPping the column. You can ALTER the column type.
    - IDENTITY does not guarantee:
        - Uniqueness - you need to use PRIMARY KEY or UNIQUE.
        - Consecutive values - IDENTITY numbers are not guaranteed to be consecutive. If any error occurs when inserting a row with an IDENTITY, that number is not reused.
- SEQUENCES - see topic 5.

5. Design and implement SEQUENCES

- SEQUENCEs can create autonumbering.
- What is the difference between SEQUENCE and IDENTITY?
    - Unlike IDENTITY, SEQUENCE is not limited to a table.
    - You can use it for multiple tables and columns.
    - You can change any DEFAULT which uses a SEQUENCE.
- You create a SEQUENCE by using:
    - CREATE SEQUENCE SequenceName
- You can optionally then add
    - AS integer_type - from tinyint to bigint, or decimal/numeric with a scale of 0
    - START WITH number
        - It must between the minimum and maximum values.
        - The default is the minimum value
    - INCREMENT BY number
        - The default is 1. It can be negative, but cannot be zero.
    - MINVALUE number or NO MINVALUE

    - The default is the minimum value for the data type
  - MAXVALUE number or NO MAXVALUE
    - The default is the maximum value for the data type
  - CYCLE or NO CYCLE
    - If the value goes to the maximum value, is the next value the minimum value or an error
  - CACHE number or NO CACHE
    - A cache retains the current value and the number of values left in the cache. This can reduce disk access.
    - However, if there is an unexpected shutdown, there may be a loss of numbers in the cache.
- SEQUENCEs can create autonumbering.
- What is the difference between SEQUENCE and IDENTITY?
  - Unlike IDENTITY, SEQUENCE is not limited to a table.
  - You can use it for multiple tables and columns.
  - You can change any DEFAULT which uses a SEQUENCE.
- You create a SEQUENCE by using:
  - CREATE SEQUENCE SequenceName
- You can optionally then add
  - AS integer_type - from tinyint to bigint, or decimal/numeric with a scale of 0
  - START WITH number
    - This could be useful if you are adding a SEQUENCE to existing data - start where the existing data ends
    - It must between the minimum and maximum values.
    - The default is the minimum value
  - INCREMENT BY number
    - The default is 1. It can be negative, but cannot be zero.
  - MINVALUE number or NO MINVALUE

    - The default is the minimum value for the data type
- MAXVALUE number or NO MAXVALUE
    - The default is the maximum value for the data type
- CYCLE or NO CYCLE
    - If the value goes to the maximum value, is the next value the minimum value or an error
- CACHE number or NO CACHE
    - A cache retains the current value and the number of values left in the cache. This can reduce disk access.
    - However, if there is an unexpected shutdown, there may be a loss of numbers in the cache.

- To get the next value in the sequence, use NEXT VALUE FOR SequenceName
- You can see all sequences using
    - SELECT * FROM sys.sequences

6. Design and implement partitioning for tables and indexes

- To implement partitioning, you would need:
    - A partition function, which uses the contents of one column to assign a partition to a particular row.
    - A partition scheme, which maps each partition to a filegroup.
- You can also add additional filegroups, and then assign data files to that filegroup.
    - This is not possible in Azure SQL Database or SQL Database in Fabric, which can only use the PRIMARY filegroup.
- To create the partition function, use:
    - CREATE PARTITION FUNCTION NameOfFunction (DataType) AS RANGE LEFT/RIGHT FOR VALUES (PartitionValues)
    - The LEFT/RIGHT indicates whether:
        - A partition goes up to a figure (LEFT), or
        - From a figure (RIGHT).
- There will be one extra partition than you have partition values. For example, for:

- CREATE PARTITION FUNCTION PartitionLEFT(Numbers INT) AS RANGE LEFT FOR VALUES (10, 20, 30)
    - The first partition will contain all values up to 10 - this includes NULL.
    - The second partition will contain values after 10 to 20 (but not including 10 itself),
    - The third partition will contain values after 20 to 30, and
    - The fourth partition will contain values after 30.
- If you are using RANGE RIGHT, then:
    - The first partition will contain all values up to, but not including, 10, including NULL.
    - The second partition will contain values from 10 to 20, not including 20,
    - The third partition will contain values from 20 to 30, not including 30, and
    - The fourth partition will contains values from 30 onwards.

- If you use NULL in your PartitionValues and use RANGE RIGHT, then the first partition will be empty.
- You then apply a partition scheme, which allows you to allocate a filegroup to a partition. Use:
    - CREATION PARTITION SCHEME NameOfScheme
    - AS PARTITION NameOfFunction
    - TO (Filegroup names, separated by commas)
- To see what partition a row is in, you can use $PARTITION.NameOfFunction(ValueUsedForPartitioning).
    - It will be an int between 1 and the number of partitions.
- You can say which filegroup will be used for the next new partition range using NEXT USED:
    - ALTER PARTITION SCHEME NameOfScheme
    - NEXT USED FilegroupName;
- You can add an additional partition by using:

    - ALTER PARTITION FUNCTION NameOfFunction ()
    - SPLIT RANGE (NewPartitionValue);
- To merge two partitions together, you can use:
    - ALTER PARTITION FUNCTION NameOfFunction ()
    - MERGE RANGE (PartitionValueToBeRemoved);
- For information about partitioning, you can query:
    - sys.partition_schemes,
    - sys.partition_functions and
    - sys.partition_range_values

7. Create views

- You can encapsulate (save) a SELECT statement as a View.
- Views can be used in a SELECT statement in place of a table (for example, in the FROM clause as a data source).
- To Create a new view, or Alter an existing view, you can use:
    - CREATE/ALTER/CREATE OR ALTER
        - CREATE OR ALTER can be used in SQL Server 2016 or later.
    - VIEW ViewName
    - AS
    - One (and only one) SELECT statement, which can include JOINs and aggregation, but cannot use:
        - An ORDER BY clause.
            - Unless you have a TOP in the Select clause. The ORDER BY clause is used to select the TOP number of rows. However, it is not necessarily used to order the end result.
        - An INTO keyword.
        - A temporary table or a table variable.
- The CREATE/ALTER View statement must be the only statement in a query batch.
    - You can use GO to end the previous query batch.
- You can use these catalog views to get information about views:

- sys.views - this contains general information about views.
- sys.columns - this includes information about each column.
- sys.syscomments - this includes the "text" column, which includes the View definition.
- sys.sql_expression_dependencies - shows when one object depends on another object.
- After the name of the view, you can then optionally specify the output columns for the view in brackets/parentheses.
    - These can be differently named than the column names in the SELECT statement.
    - All columns in a VIEW must have a column name - so if you have a mathematical computation, it will need an alias.
    - Column names cannot be duplicated in a view. For example, if you have SELECT * FROM tblOne JOIN tblTWO..., this will probably produce duplicate column names.
- You can then optionally have:
    - WITH ENCRYPTION. If you use this option, you cannot read the contents of the view in sys.syscomments - the text column is NULL.
    - WITH SCHEMABINDING. With this option:
        - The base table(s) cannot be modified so that the view definition would be changed.
            - This means that some changes can be made, but not all changes.
            - This includes dropping any tables or views which are referred in this query
        - When referring to tables, other views or user-defined functions, you must use the schema.object naming system.
        - You cannot use a * in the SELECT clause
    - WITH VIEW_METADATA. When APIs (or similar) ask for metadata (data about data, such as the structure) for a view, they will only find metadata about the view and not about the tables/views they refer to.

- You can use UPDATE, INSERT and DELETE statements on Views, as long as:
    - Modifications use only one base table's columns.
        - You cannot do DDL commands against multiple base tables at once.
        - If you want to modify multiple tables, then you could use an INSERT OF trigger.
    - The columns do not use aggregations or computations.
    - They are not affected by GROUP BY, HAVING or DISTINCT clauses.
    - You do not use both TOP (in the select clause) and WITH CHECK OPTION.
- At the end of the INSERT/ALTER statement, before the optional semicolon, you can add WITH CHECK OPTION.
    - If you use this, you would not be able to INSERT/UPDATE data in a view that could not be accessed afterwards in the view.
    - For example, if the view includes WHERE ID<130, if you use WITH CHECK OPTION, then you cannot Insert data with an ID of 130 or above.

8. Create scalar functions

- Scalar functions return a single value.
- They can be used:
    - in a SELECT statement,
    - in a CHECK constraint on a column, and
    - in a computed column.
- The syntax is:
    - CREATE/CREATE OR ALTER FUNCTION FunctionName
    - The parameters in brackets/parentheses, including:
        - an optional AS,
        - the data type, and
        - an optional = default value
    - RETURNS DataType,
    - Optional function options, including

    - WITH ENCRYPTION (see topic 7)
    - WITH SCHEMABINDING (see topic 7)
    - WITH EXECUTE AS Person (see topic 10),
    - WITH RETURNS NULL ON NULL INPUT.
        - If any argument contains a NULL, then the response will be NULL - the function is not called.
        - The opposite (and default) is WITH CALLED ON NULL INPUT.
  - An optional AS,
  - BEGINS
  - The function itself, ending with RETURN value
  - END
  - an optional semi-colon
- You cannot use BEGIN/END TRY and CATCH in a function.
- Information about a scalar function can be found in sys.sql_modules.

9. Create table-valued functions

- Table-valued functions return a table.
- A table-valued function starts with:
  - CREATE/CREATE OR ALTER FUNCTION FunctionName
  - The parameters in brackets/parentheses, including:
    - an optional AS,
    - the data type, and
    - an optional = default value
- An inline table-valued function continues with:
  - RETURNS TABLE
  - The optional function options,
  - An optional AS
  - RETURN (The SELECT statement)

- A multi-statement table-valued function would continue:
    - RETURNS @NameOfTable TABLE
    - (Definition of the table)
    - AS BEGIN
    - The SELECT statements
    - RETURN
    - END
- The function can be used where you use tables (for example, in the FROM clause of a SELECT statement).
- You can also use in conjunction with other tables. However, instead of using a JOIN, because you are applying to a single row at a time, you use either:
    - CROSS APPLY - this returns the table where the results of the function is not null, and
    - OUTER APPLY - this returns the entirety of the table, together with the results of the function.

10. Create stored procedures

- A stored procedure run one or more T-SQL statements. It can:
    - Use input parameters,
    - Send output parameters,
    - Run statements, including calling other procedures,
    - Return a status value to show success/failure.
- The benefits of stored procedure include:
    - Statements are run as a single batch of code,
    - Use impersonation, so that the statements are run as a different user,
    - Allow code to be reused and maintained more easily,
    - Because an execution plan is created after the first time it has been run, on subsequent runs it uses the same execution plan. This means that the execution plan need not be recreated.
- To create a stored procedure, use:
    - CREATE/CREATE OR ALTER PROC/PROCEDURE ProcedureName

    - You can then add Parameters.
        - Parameter names start with an @ sign,
        - It needs to be followed by the data type.
        - You can then add an equal sign and a default value. If you do this, you do not need to provide a value for the parameter.
        - If it is an output parameter, it should be followed by OUT or OUTPUT.
        - Multiple parameters need to be separated by a comma.
    - You can then optionally add:
        - WITH RECOMPILE - The execution plan will be recompiled each time. This can be useful if using different parameter values will give different execution plans.
        - WITH ENCRYPTION - the original text will be hidden (but is visible if you directly access database files)
        - WITH EXECUTE AS SecurityName. This includes SELF, OWNER and a 'UserName'.
    - AS [Statement], or AS BEGIN [Statement(s)] END
    - You can optionally end with a semicolon.
- As one of the statements, you can use SET NOCOUNT ON to avoid the message "(X row(s) affected)".
- To run a stored procedure, use EXEC or EXECUTE StoredProcedureName, followed by the parameters.
    - You can use the Parameter name or just the value.
- To use an OUTPUT parameter:
    - You add OUTPUT in the CREATE PROC line,
    - You can use SET or SELECT @Parameter = to assign the parameter,
    - You use OUTPUT in the EXEC line after the Parameter.
- When you execute a stored procedure, you can also access the return status.
    - You can access it by using:
        - DECLARE @Result INT

    - EXEC @Result = StoredProcedureName ...
- You can set the return status using RETURN.
- The execution of the stored procedure then ends. Any statements after RETURN are not run.

11. Create triggers

- Triggers can run when an INSERT, UPDATE or DELETE (DML) statement runs on a table/view.
    - You can also add triggers based on DDL events (CREATE, ALTER and DROP), and based on the LOGON event (when a user logs in).
- There are two different types of when the trigger will fire:
    - AFTER (or FOR). This runs after the event has completed.
    - INSTEAD OF. For DML triggers only, this runs instead of the event.
        - This cannot be used on views using WITH CHECK OPTION.
        - You cannot use INSTEAD OF with DELETE/UPDATE if there is a relationship using ON DELETE/UPDATE CASCADE.
- It must be the first statement in the batch and only applies to one table/view in the current database.
- You can only have one INSTEAD OF INSERT trigger for a table/view.
- The syntax is as follows:
    - CREATE/CREATE OR ALTER TRIGGER TriggerName
        - For DML triggers: ON table/view
        - For DDL triggers: ON ALL SERVER or ON DATABASE
        - For LOGON events: ON ALL SERVER
    - FOR/AFTER/INSTEAD OF (for DML triggers only)
    - For DML triggers: INSERT, UPDATE, DELETE
        - You can use 1 to 3 of these.
        - A TRUNCATE TABLE statement does not activate a DELETE trigger.
    - AS - and then the Select Statement(s).
        - If you want to use more than one Select statement, then use AS BEGIN ... END.

        - These can use the "deleted" and "inserted" logical/conceptual tables.
        - An UPDATE statement will result in both the delete table, which contains the old version, and inserted table, which contains the new version.
- Before the word "AS", you can optionally have, for DDL triggers:
    - WITH ENCRYPTION
        - Means that the CREATE TRIGGER statement text is obscured.
        - It cannot be published when using SQL Server replication.
    - WITH EXECUTE AS [Clause]
        - This allows you to say which security context the triggers is being executed under.
        - This is needed for memory-optimized tables.

12. Write common table expressions (CTEs)

Non-recursive CTEs

- A common table expression (CTE) allows you to use the results of a query in another query without using a view.
    - CTEs can be used in, among other places, views.
- To create a CTE, you use this syntax:
    - WITH CTEName AS (SELECT statement)
    - SELECT/INSERT/UPDATE/MERGE/DELETE statement…
    - FROM CTEName.
- You cannot use with a CTE:
    - ORDER BY (using you have a TOP or similar clause). If you want to use an ORDER BY, use it outside of the CTE.
    - INTO
- If you want to use additional CTEs, you can add them after the (SELECT statement) by adding a comma and then the next CTEName.
    - These additional CTEs can refer to earlier (not later) CTEs.
- You cannot nest CTEs.

    - So you cannot have WITH CTEName AS (WITH CTEName2 AS (...
- You can optionally add after the CTEName the output column names.
    - Duplicate output names within a CTE are not allowed.
- If you are running multiple statements, and one of them (not the first) is a CTE, then the previous statement must end with a semicolon.

UNION, UNION ALL, EXCEPT and INTERSECT operators

- UNION and UNION ALL produces a dataset which includes all of the rows from two SELECT statements.
    - UNION removes duplicates,
    - UNION ALL retains duplicates.
- Do not end the first SELECT statement with a semicolon.
- You can then use UNION/UNION ALL with additional SELECT statements.
- The number of columns and the order must be the same in both SELECT statements, and the data types must be compatible.
- The resulting column names are taken from the first SELECT statement.
    - It ignores the column names from the second SELECT statement; it does not even require column names.
- INTERSECT retrieves distinct rows which are the same in both SELECT statements.
- EXCEPT retrieves distinct rows which are in the first SELECT statement but not in the second SELECT statement.
    - EXCEPT is the only one of the 4 operators where the order of the SELECT statements is important.
- The requirements for EXCEPT and INTERSECT are the same as for UNION and UNION ALL.

Recursive CTEs

- You can create a Recursive CTE which builds an overall response.
    - If it is part of a batch, the previous statement must end with a semicolon.
- It contains:
    - At least one anchor member. This defines the start of the data set.
    - At least one recursive member.

- The anchor member are rows which are the first members of the resulting dataset.
- The recursive members are rows which are based, in part, on the anchor members.
- Once the recursive members have been worked out, they will then be added to the resulting dataset.
- The recursive members can then be recalculated, and so on.
- The anchor member is connected to the recursive member using a UNION ALL.

13. Write queries that include window functions

ROW_NUMBER, RANK, DENSE_RANK and NTILE

- You can use these ranking functions:
    - ROW_NUMBER - this numbers the rows consecutively,
    - RANK - this acts as ROW_NUMBER, but ties will have the same number. It will then skip any values to get up to the ROW_NUMBER.
    - DENSE_RANK - this acts as RANK, but it will not skip any values.
- After the function, add () OVER(). In the OVER(), you add:
    - PARTITION BY ColumnName(s)
        - This is optional, and divides the data into different partitions.
        - For these functions, it restarts the numbering.
    - ORDER BY ColumnName(s).
        - This is required can include ASC/DESC.
        - This determines the order for the numbering.
        - If the ORDER BY does not uniquely define the order, then the results will not necessarily be in the same order when you re-run the statement.
- You can also use the NTILE function.
    - This divides the rows into X divisions. You use NTILE(X) OVER(...).
- If you are using SQL Server 2022 or later, you can use the WINDOW clause.
    - WINDOW WindowName AS (PARTITION BY ... / ORDER BY ... / ROW/RANGE ...)

        - By defining the WindowName, it means that you do not need to repeat the definition multiple times.
    - This WINDOW clause goes between the HAVING and ORDER BY clauses.
    - You can then use OVER WindowName.

LAG and LEAD

- You can also use these analytic functions:
    - LAG and LEAD.
        - This get the value of a column/calculation in the previous/next row(s).
        - In the brackets/parentheses after LAG/LEAD, it uses the following arguments:
            - The expression that you want to calculate.
            - The offset number of rows. This is optional, and the default is 1.
            - The default value to be shown if the calculation is after/before the partition. This is optional, and the default is NULL.
        - Outside of the brackets/parentheses, you can then optionally add:
            - IGNORE NULLS to ignore rows which contain NULLs, or
            - RESPECT NULLS to include rows which contain NULLS. This is the default.
        - These functions then use an OVER clause with optionally PARTITION BY and you must have ORDER BY.

FIRST_VALUE and LAST_VALUE

- FIRST_VALUE and LAST_VALUE.
    - This returns the first/last value in a partition/data set.
    - It uses:
        - one argument (the expression), together with
        - (optionally) IGNORE/RESPECT NULLS, and
        - the OVER clause with (optionally) PARTITION BY and you must have ORDER BY.

- It also uses a rows/range clause.
    - This limits the rows in a partition. The syntax is ROWS/RANGE BETWEEN FirstRow AND LastRow.
    - The row definition can be:
        - UNBOUNDED PRECEDING/FOLLOWING - the beginning/end of the partition.
        - 1 (or another number) PRECEDING/FOLLOWING
        - CURRENT ROW.
    - The default is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
    - The difference between RANGE and ROW is that:
        - ROW evaluates each row individual, whereas
        - RANGE includes all rows which tie with the same value. For example, CURRENT ROW would include all rows with the same current value.

CUME_DIST and PERCENT_RANK

- CUME_DIST. This shows the cumulative distribution of a row.
    - If there are 6 rows, then:
        - the first row will have a CUME_DIST of 1/6,
        - the second will have 2/6, and
        - then last 1.
    - It uses the OVER clause with the (optionally) PARTITION BY and needs the ORDER BY.
- PERCENT_RANK. This shows the rank of a row.
    - If there are 6 rows, then:
        - the first row will have a PERCENT_RANK of 0/5,
        - the second will have 1/5, and
        - then last 1 (which is 5/5).
    - It uses the OVER clause with the (optionally) PARTITION BY and needs the ORDER BY.

PERCENTILE_CONT and PERCENTILE_DISC

- PERCENTILE_CONT.
  - This calculates a value based on a percentage.
  - Its syntax is:
    - PERCENTILE_CONT(PercentageValue) - if the PercentageValue is 0.5, it will calculate the median value.
    - WITHIN GROUP (ORDER BY Column(s))
    - OVER () - You can use PARTITION BY within the OVER.
  - You would often a DISTINCT in the SELECT statement, as there is no GROUP BY clause.
- PERCENTILE_DISC.
  - This is similar to the PERCENTILE_CONT, except it returns the value which is at the PercentageValue.
  - If there is no such value, then it will return the value which is below the PercentageValue.

- You can also use aggregate functions with an OVER clause.
  - For example, SUM, COUNT, AVG, MAX, MIN.
  - The SUM, AVG, MAX and MIN functions use in the OVER:
    - An (optional) PARTITION BY and
    - An ORDER BY. While it is technically optional, you probably should include it.
    - An optional RANGE/ROW function.

Write advanced T-SQL code

14a. Write queries that include JSON functions: JSON_OBJECT

- To convert text to JSON, you can either create the JSON format yourself, or you can use the JSON_OBJECT function:
- The data is returned in the nvarchar(max) data type.
  - If you need it to be returned in the json data type, then add before the closing bracket/parenthesis: RETURNING json

- You can also use, just before the close bracket/parenthesis:
    - NULL ON NULL – if the input value is NULL, then json value will be NULL. This is the default setting.
    - ABSENT ON NULL – if the input value is NULL, then neither the key nor the value will be used in the resulting JSON.

14b. Write queries that include JSON functions: JSON_ARRAY

- If you are converting text to a JSON array (where there CAN be more than one object), then you can use JSON_ARRAY.
    - This is often used in conjunction with JSON_OBJECT.
- Again, you can also use, just before the final close bracket/parenthesis:
    - NULL ON NULL to retain NULLs in the end JSON – this is the default, or
    - ABSENT ON NULL – this omits the element if the value is NULL.
    - RETURNING json – to return a json data type instead of an nvarchar(max) data type.

14c. Write queries that include JSON functions: JSON_ARRAYAGG

- The JSON_ARRAYAGG function returns an array from a query.
    - It is available in SQL Server 2025 or later (together with Azure SQL Database, Fabric SQL Database and so on).
- At the end of the JSON_ARRAYAGG function (before the closing bracket/parenthesis), you can add:
    - ORDER BY FieldName
- Again, you can also use, just before the final close bracket/parenthesis:
    - NULL ON NULL to retain NULLs in the end JSON – this is the default, or
    - ABSENT ON NULL – this omits the element if the value is NULL.
    - RETURNING json – to return a json data type instead of an nvarchar(max) data type.

14d. Write queries that include JSON functions: JSON_CONTAINS

- If you want to see if there is a certain value in a JSON document, you can use JSON_CONTAINS.
- At the time of recording, it is only available in SQL Server 2025.

    - Not in Azure SQL Database or Fabric's SQL Database.
- The search term can be a scalar value (string/number, for example) or a json data type value.
- It returns 1 if the value is found, and 0 if it is not found.
    - It also returns NULL if any of the arguments is NULL.
- You can also use a third optional argument which contains the path.
    - The default is '$', which is stated as being the entire document.
        - However, this does not work for finding scalar values (as opposed to json vales). Instead, I would use '$.*' to search the entire document.
    - If you wanted to search the object Given name, then you could use:
        - SELECT JSON_CONTAINS(@jsondocument, 'Phillip', '$."Given name"')
- You can also use a fourth, optional, argument which contains the search mode.
    - A 0 indicates that the match must be exact. For example, searching for 'Phil' which not find 'Phillip'. This is the default
    - A 1 indicates that the search uses a LIKE, using the % and _ wildcards.

14e. Write queries that include JSON functions: OPENJSON

- OPENJSON parses JSON documents.
    - This needs SQL Server 2016 or later, or Azure SQL Database and Fabric SQL Database and other services.
- It can be used in the FROM clause of a document:
    - SELECT * FROM OPENJSON(@jsondocument)
- It retrieves:
    - The key number - nvarchar(4000),
    - The value - nvarchar(max), and
    - The type
        - NULL = 0,
        - String = 1,
        - Number = 2,

    - Boolean = 3,
    - Array = 4,
    - Object = 5.

- If you do not want the entirety of the document, you can use an optional second argument to specify the path.
- If you want the content to be converted into rows and columns (which is called "shredding"), then you need to use a WITH after the OPENJSON() function, specifying the format:
    - WITH (Column1 DataType1 Location1,
    - Column2 DataType2 Location2)

14f. Write queries that include JSON functions: JSON_VALUE

- If you want to retrieve a value from a JSON document, you can use JSON_VALUE.
    - This can be used in SQL Server 2016 and above.
- You specify:
    - The JSON document, and
    - The path.
- In SQL Server 2025 and above, you can also use an optional third argument, RETURN DataType, to return the value as the specified data type.
- For example, to retrieve the first Given Name, you would use:
    - SELECT JSON_VALUE(@jsondocument, '$[0]."Given name"')
    - $ indicates the root.
    - [0] indicates the first item - it is zero-based.

15. Write queries that include regular expressions

- Regular expressions allow you to see for a range of characters. They include:
    - . = any character
    - [abc] - any of these characters
    - [^abc] - any character apart from these characters
    - \d or [0-9]- any digit
    - \D or [^0-9] - a character apart from a digit

- \w - any digit, lower/upper-case letter, or underscore (not spaces)
- [A-Za-z] - any lower or upper-case letter
- [^A-Za-z] - any character apart from a letter
- x* - zero or more "x"s,
- x+ - one or more "x"s,
- x? - zero or one "x"s,
- x{n,m} - n to m "x"s,
- x{n,} - n or more "x"s,
- x{n} - n "x"s.
- ^ at the beginning means "at the beginning of text or line".
- $ at the beginning means "at the end of the text or line".

- You can also use the following flags:
  - i - case-insensitive,
  - c - case-sensitive (the default).

15a. Write queries that include regular expressions, such as REGEXP_LIKE

- The regular expression functions were introduced in SQL Server 2025 (Compatibility Level 170).
- REGEXP_LIKE looks through a string for a regular expression.
  - The first argument is what you are searching through, or
  - The second argument is what you are looking for,
  - The optional third argument are flags. The default is 'c'.

15b. Write queries that include regular expressions, such as REGEXP_REPLACE

- REGEXP_REPLACE replaces one regular expression with another.
  - The first two arguments are what you are searching through and looking for.
    - You can add brackets/parentheses in the second argument to divide the regular expression.

- The optional third argument is what you want to replace it with.
    - You can use \1 to use the first regular expression part.
- The optional fourth argument is the position it should start at. The default is 1.
- The optional fifth argument is which occurrence should be replaced. The default is 0, which means all occurrences.
- The optional sixth argument is the flags. The default is 'c'.

15c-e. Write queries that include regular expressions, such as REGEXP_SUBSTR, REGEXP_INSTR, and REGEXP_COUNT

- REGEXP_SUBSTR extracts part of a string.
    - The first to fifth arguments are the same as the first, second, fourth, fifth and sixth arguments in the REGEXP_REPLACE function (there is no argument for a replacement string).
    - The sixth optional argument shows which bracketed group to return.
- REGEXP_INSTR find the position of the regular expression.
    - The first character position is 1.
    - If the regular expression cannot be found, this function will return 0.
    - The first to fourth and sixth and seventh arguments are the same as the first to sixth arguments in the REGEXP_SUBSTR function.
    - The optional fifth argument is whether you want to start searching at the beginning (0) or at the end of the letter (1). The default is 0.
- REGEXP_COUNT finds the number of times that the regular expression is matched.
    - The first two arguments are as the REGEXP_LIKE function.
    - The third function is where the searching should start.
    - The fourth function is the flags.

15f-g. Write queries that include regular expressions, such as REGEXP_MATCHES and REGEXP_SPLIT_TO_TABLE

- REGEXP_MATCHES returns a table which contains a row for each regular expression.

    - The arguments are as the REGEXP_LIKE function.
    - The columns which are returned are:
        - match_id - an incremental number,
        - start_position - the start position of the regular expression. The first position is 1.
        - end_position - the end position.
        - match_value - the expression which has been matched,
        - substring_matches - a JSON expression containing the value, start and length.
- REGEXP_SPLIT_TO_TABLE splits based on a delimiter
    - The arguments are as the REGEXP_LIKE function.
    - The columns which are returned are:
        - value - the value which is returned, and
        - ordinal - which regular expression is being returned

16a. EDIT_DISTANCE

- Fuzzy string matching functions were introduced in SQL Server 2025.
- EDIT_DISTANCE calculates how many insertions, deletions and substitutions are needed to go from one string to a second string.
    - It currently does not support transpositions.
- The syntax is EDIT_DISTANCE(CharFrom, CharTo).
    - If any of the arguments is NULL, the result will be NULL.
    - The arguments cannot be [n]varchar(max).
    - You can also have a third argument, MaximumDistance. This limits the answer to the actual Edit_Distance or the MaximumDistance, whichever is lower.

16b. EDIT_DISTANCE_SIMILARITY

- EDIT_DISTANCE_SIMILARITY gives a result from 0 (no match) to 100 (complete match).
    - It currently does not support transpositions.
- The syntax is EDIT_DISTANCE_SIMILARITY(CharFrom, CharTo).

- It is calculated as 100% less
    - The EDIT_DISTANCE divided by the longest string length times 100.

16c. JARO_WINKLER_DISTANCE

- JARO_WINKLER_DISTANCE calculates the edit distance between two strings using the Jaro-Winkler edit distance algorithm.
- This focuses more on the prefix of the strings. It goes from 0 (complete match) to 1 (no match).
- You can also use JARO_WINKLER_SIMILARITY, to calculate it as a percentage, where 0 is no match and 100 is a complete match.
    - It is calculated as 100% less 100% times JARO_WINKLER_DISTANCE.

17. Write graph queries that use the MATCH operator

- The MATCH operator works with graph nodes and edge tables.
    - It is available in SQL Server 2017 or later.
- Terminology:
    - A graph is a collection of nodes and edge tables.
    - A node is an item.
    - An edge table is a relationship between two nodes.
- You can use MATCH to go from nodes through edge tables to other nodes.
- To create a node table, add AS NODE at the end.
    - This creates the table with a column starting with $node_id to identify each node.
- To create an edge table, add AS EDGE at the end.
    - This creates the table with the following additional columns:
        - $edge_id,
        - $from_id and $to_id to go between the nodes.
- The syntax for MATCH is:
    - MATCH(Node1-(EdgeTable)->Node2) to start at Node 1, or
    - MATCH(Node1<-(EdgeTable)-Node2) to start at Node 2.
    - You can add additional Edges and Nodes.

18. Write correlated queries

- A derived table is generally used in the FROM or WHERE clause.
    - It is a second (inner) query inside a first SELECT statement (outer query/select).
    - This second query can be run independently from the first statement.
    - If used with an IN in the WHERE clause, it can act as an INNER JOIN.
    - You can use a WHERE, GROUP BY and/or HAVING clause (but not an ORDER BY clause unless TOP is also used in the SELECT clause).
    - Derived tables can also use derived tables (up to 32 levels deep!).
    - They can be used with UPDATE, DELETE and INSERT statements.
- You can also use the results of a query (a subquery) in the SELECT or WHERE clauses.
    - It would have a single row and a single column.
    - A derived query in the FROM clause, which acts as a data source, should have an alias.
    - The derived query in the WHERE clause can be used in a condition.
- A correlated query can be used in the SELECT clause.
    - This is a second (inner) query inside a first (outer) SELECT statement.
    - For the SELECT clause:
        - the results of the inner query needs to have one column and one row.
        - It cannot use GROUP BY or HAVING clauses, as it needs to result in one row.
        - It can be used to replace a LEFT JOIN.
    - The "correlated" part of the query means that it interacts with the main query.
    - The inner query uses a WHERE condition which refers to the outer query.
- A correlated query can also be used in the WHERE clause.
    - It is often used with:
        - WHERE EXISTS (SELECT * FROM ...)

    - You can also use SELECT 1 with EXISTS, as it is looking for the presence of a row (true/false), not the contents. However, SELECT * is more usual.

- WHERE ColumnName IN (SELECT ColumnName FROM tblCorrelated AS C WHERE C.ColumnName = O.ColumnName)
    - Where "O" is the alias of the Outer query.

19. Implement error handling

- To implement error handling in code or a Stored Procedure, you can use:
    - BEGIN TRY...END TRY for the code, and
    - BEGIN CATCH...END CATCH for the code to run if there is an error.
- If there is an error, you can use:
    - ERROR_NUMBER and ERROR_MESSAGE - the error number and message,
    - ERROR_PROCEDURE and ERROR_LINE - the name and line of the procedure/trigger where the error happened,
    - ERROR_SEVERITY - the typical severity is 16.
    - ERROR_STATE - a state code, which is different for each error raised.
- Alternatively, you can test for things which would produce errors before they happen.
    - You can use IF Condition ... ELSE ... to test for a condition, and do different code based on whether the condition is true or false.
        - The ELSE is optional. The code after the END is run if the condition is false.
        - You do not use an END at the end.
        - However, if you want to use more than one statement in a true/false block, you need to use BEGIN...END to encompass the block.
- You should also beware of using SELECT * - the columns in the results can be different, based on the tables/queries.
    - Similarly, you should avoid using ORDER BY 1 (column number), as the order may also change.

Design and implement SQL solutions by using AI-assisted tools

20. Interpret security impact of using AI-assisted tools

- For Microsoft Fabric:
    - The user's prompts and additional data is sent to the Azure OpenAI service.
        - This is not the public version of OpenAI, but Azure's version.
        - The data is only data that the current user is able to access.
        - It might include dataset schema, data points, and other relevant information.
    - The outputs are only visible to that user (unless you choose to share it)
    - Data is not used to train models.
    - Data is not available to other customers.
    - Prompts are not retained.
    - Data processed by Fabric's Copilot remains within the capacity's geographic region, if there is an Azure OpenAI service in that region.
        - Administrators can override this.
            - This can be useful if Azure OpenAI is not available, or availability is useful, in your region.
            - To do this, go to Admin Center - Tenant settings - and turn on "Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance."
- For Github Copilot:
    - Prompts go through a GitHub proxy for pre-inference checks (screening for language, relevance, and hacking/jailbreaking).
    - Data is encrypted in transit and at rest.
    - Data is not used to train models.
    - Code is not "copied and pasted" from a codebase. Microsoft has an Intellectual Property indemnification policy, so if you face a copyright

claim due to an unmodified suggestion from GitHub Copilot, Microsoft will defend you in court.

- For Business/Enterprise customers, data is retained:
    - User Engagement Data (pseudonymous identifiers from user interactions) - for 2 years,
    - Feedback data (user feedback, thumbs up/down, and support ticket feedback) - for as long as needed,
    - Prompts and Suggestions - not retained for Chat and Code Completions, and 28 days for other access.
    - For Copilot Coding Agent, session logs are retained for the account's lifetime.
- Data is indexed. You can configure content exclusions for the repository/organization to stop sensitive files from being indexed.
- The website GitGuardian has the following concerns:
    - Potential leakage of secrets and private code.
        - It says that 6.4% of sampled repositories have leaked at least one secret.
        - This is due to:
            - potential security vulnerabilities from models
            - sloppy writing with respect to code quality/security, and
            - outdated versions of software.
    - Insecure Code Suggestions
        - Copilot is trained on code which is not 100% secure.
    - Poisoned data making malicious code
        - Data that GitHub Copilot is trained on might be deliberately unsecure.
    - Hallucinations
        - Copilot making up packages.
    - Lack of Attribution and Licensing

    - Could Copilot be trained on copyrighted code, and then use that code for your code?

21. Enable GitHub Copilot and Microsoft Copilot in Fabric

- GitHub Copilot allows you to create T-SQL more quickly in SQL Server Management Studio (SSMS) 22+.
- To install it, either:
    - In SQL Server Management Studio:
        - Click on the dropdown in the top-right hand corner next to the GitHub icon and click on Install Copilot,
        - Click on Install.
    - Open the "Visual Studio Installer":
        - Click on Modify next to SSMS,
        - Check "AI Assistance",
        - Click "Close".
- Then, after asking a question, you can sign into GitHub in SSMS using either a GitHub or Google login.
- To create a GitHub account:
    - Go to https://www.github.com
    - Enter your email address and click Sign up.
    - Enter a new user name and password.
    - You can then verify your account.
- You can use GitHub Copilot to:
    - Chat - ask questions about the database, get help about T-SQL and create code.
    - Code completions - this happens when you write T-SQL. New code is shown in grey.
    - Next Edit Suggestions - suggests edits based on changes.
    - Database instructions - store business rules and context in your database.
- Note: there are limitations to how much you can use GitHub Copilot.

    - From 1 June 2026, this is based on GitHub AI Credits, which are priced at 1 US cent each.
    - Interactions may take a single, multiple or part of a credit, based on the model and number of tokens consumed.
- GitHub Copilot is also integrated with the MSSQL extension for Visual Studio Code.
- To enable Microsoft Copilot in Fabric, the administrator needs to:
    - go to Settings - Admin portal - Tenant settings, and
    - Enable "Users can use Copilot and other features powered by Azure OpenAI".
    - You can change the "Apply to" to:
        - The entire organization, or
        - Specific security groups.
        - You can also select "Except specific security groups".
    - You may also need to enable:
        - Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance
        - Enabled for the entire organization
- The Fabric capacity needs to be a region which supports Copilot for all Fabric workloads, and the Fabric capacity needs to be running.
    - There are some regions where it is only supported for Power BI.
- The workspace needs to use a Fabric capacity of at least F2.
    - In the workspace, go to Workspace settings,
    - Then go to the Workspace type.
    - Click Edit, and select either Fabric or Fabric Trial.

22. Configure model and Model Context Protocol (MCP) tool options in a GitHub Copilot or Copilot in Fabric chat session

- In SQL Server Management Studio (SSMS), in the GitHub Copilot chat, you can click on the model at the bottom and change it.

        - You can also click on "Manage Models" and choose an alternative model from Anthropic, Google, OpenAI, xAI, Azure and Foundry Local.
- In Visual Studio Code:
    - in the Chat, you can click on the model (by default, the "Auto"), and select:
        - A different model,
        - An "Other model", or
        - Manage Models..., and click on "+Add Models...", and select an alternative model from Anthropic, xAI, Google, OpenRouter, OpenAI, Ollama or Azure.
    - You can add a Model Context Protocol (MCP) by:
        - Opening the extensions panel (on the left-hand side),
        - Enter mcp.
        - Find the relevant MCP server and click on Install.
    - To list the MCP servers:
        - Open the command palette (Ctrl/Command+Shift+P)
            - Or clicking on the bar at the top-middle of the screen and press >.
        - Enter "MCP: List Servers".
    - There you can configure an MCP server by clicking on it and selecting an option.
        - You can also see various options by going to the Extension page and reading the "Getting Started" page.
- There is currently no way to configure the model or MCP tool options in a Fabric Copilot chat session.
- However, see topic 24.

23. Create and configure GitHub Copilot instruction files

- You can create Copilot instruction files by creating a .github/copilot-instructions.md file.
    - These instructions apply to all chat requests in the workspace (project-wide instructions).

    - It can start with an optional name and description, which will be the description shown on hover in Chat view.

- An easy way to create this file is to ask Chat in Visual Studio Code "Create a .github/copilot-instructions.md file", or enter /generateInstructions.
- You can then alter it as required.
    - Keep individual instructions short and self-contained.
    - Explain the instruction's reasoning.
    - Use preferred and avoided patterns.
    - Add non-obvious rules (no need to add rules which are already done).
- The Claude format instructions go in the .claude/rules folder

- In Visual Studio, you may need to enable the feature:
    - Go to Tools - Options.
    - In All Settings - GitHub - Copilot - Copilot Chat section, check "Enable custom instructions to be loaded from .github/copilot-instructions.md files and added to requests".

23. Create and configure GitHub Copilot instruction files

- You can also add user-level preferences which apply to all Copilot sessions across projects.
    - They are saved in the file "copilot-instructions.md" in your User Profile (this is the parent folder to the Documents/My Documents folder).
- You can also create additional .github/instructions/*.instructions.md files which apply to specific file types or tasks.
    - Add an applyTo property at the front of the instructions, to say what files/folders the instructions apply to.
    - For example:
        - ## C# Instructions
        - ---
        - name: 'Python Standards'
        - description: 'Coding conventions for Python files'
        - applyTo: `**/*.cs`

    - ---

- You can also create the instructions by opening the Command Palette (Ctrl/Command+Shift+P) and enter "Chat: Configure Instructions & Rules...".
- You can also create reusable prompt files, by:
    - creating the prompt, and
    - saving it in the folder .github/prompts folder, with the .prompt.md extension, or entering /savePrompt.
- To access the prompt:
    - Enter #prompt or /prompt in a chat input,
    - click the + icon to add it as context.
- In a Microsoft Fabric agent, you can also go to the Explorer - Setup tab to add:
    - Agent instructions (for the entire agent) - this is also accessible from the Home menu, and
    - for each data source, Data source instructions and Example queries.
- You can test the agent in the right-hand most pane.
- You can then go to Home - Publish:
    - You can add a description of purposes and capabilities
        - This is important. When connected to other agents, it shows when that agent should go to this agent to help answer a question.
    - Publish it to Microsoft 365 Copilot.
- You can then go to Home - Settings (the Wheel) - Model Context Protocol, and copy the MCP server URL for use in conjunction with other agents.
    - In Visual Studio Code, press Ctrl/Command+Shift+P, and go to MCP: Add Server - HTTP and paste the URL.

24. Connect to MCP server endpoints, including Microsoft SQL Server and Fabric lakehouse

- You can create a data agent which you can use to create a MCP server in Fabric.
- In a Fabric-enabled workspace:
    - Click on "+New item",
    - Select "Data agent",

    - Give it a name and click Create.
    - In the Explorer - Data tab, or in the Home bar, click on "Add Data" - Data source, and add a data source.
        - This includes data in a Fabric lakehouse.
    - Select the tables in the data source you want the agent to access.
    - In the Setup tab, you can add "Data source instructions".
- To connect to an Azure SQL Database from Visual Studio Code:
    - Install the Azure MCP Server extension.
    - Sign into Azure, and select subscription, Azure SQL Server and Database.

Implement data security and compliance

25. Design and implement data encryption, including Always Encrypted and column-level encryption

- Always Encrypted encrypts the data within client applications.
    - This prevents users like administrators who manage the data to not have access to the data,
    - while still allowing people who own the data to view it.
    - You can use it in Azure SQL Database, Managed Instance and SQL Server databases.
- You will need to create two keys:
    - A column encryption key - to encrypt the data,
    - A column master key - this encrypts the column encryption keys.
- Column master keys should be stored outside of the database
    - For example, Azure Key Vault, Windows certificate store, or a hardware security module (HSM).
- To do this in SSMS:
    - Right-hand click on the table,
    - Go to Always Encrypted Wizard...
    - Click Next twice.

- Always Encrypted with Secure Enclaves allows for in-place encryption and richer confidential queries.

- Select the columns to be encrypted and select the encryption type and encryption key.
  - Deterministic encryption means that the same value will be encrypted the same.
    - Advantages: You can do point lookups (= or IN), equality joins, GROUP BY, DISTINCT, and indexing
      - Because the values are encrypted the same, searches can take advantage of indexing.
    - Disadvantages: Anyone viewing this column can see which rows have the same value, and may be able to make guesses about the data.
  - Randomized encryption - the same value will be encrypted differently.
    - Advantages: This is more secure - you cannot see similarities between similar values.
    - Disadvantages: You cannot use searching, grouping, indexing or joining.
  - It is not supported for columns which need values to be not encrypted or cannot be encrypted - for example:
    - Data types xml, timestamp, rowversion, image, ntext, text, sql_variant, hierarchyid, geography, geometry, vector, alias, user-defined types.
    - FILESTREAM columns,
    - Columns with IDENTITY or ROWGUIDCOL properties.
    - Columns included in full-text indexes (see topic 60).
    - Computed columns, spare or partitioning columns.
    - Columns with default or check constraints.
  - For randomized encryption:
    - Keys used for indexes,

        - Columns referenced by statistics, unique or foreign key constraints.

    - This uses the CREATE COLUMN ENCRYPTION KEY command.

- The columns will then be assessed.
    - It is suggested that you review the messages.
    - You can view/copy/export the report.
- In the "Master Key Configuration" page, you can then configure a Master Key.
    - This uses the CREATE COLUMN MASTER KEY command.
    - You can choose to store the Master Key in either your Windows certificate store or in the Azure Key Vault.
- If you are using an enclave, then you will be able to adjust the in-place encryption settings.
- In the "Run Settings" page, you can:
    - Run offline,
    - Run online,
    - Generate a PowerShell script to run later, and/or
    - Enable verbose logging.

- You can then encrypt the columns. To do this, you need these permissions:
    - ALTER ANY COLUMN MASTER/ENCRYPTION KEY,
- Additionally, and to query any encrypted columns, you need:
    - VIEW ANY COLUMN MASTER/ENCRYPTION KEY DEFINITION.
- You can see permissions by right-hand clicking on the database, going to Properties and Permissions.
- Once the table is encrypted, you can query the table.
    - Notice that there are similar category results.
- If you disconnect and reconnect going to:
    - Advanced - Enable Always Encrypted = True.

- and then open a new query and go to Query - Query Options - Execution - Advanced - and check Enable Parameterization for Always Encrypted, then you can use queries such as:
    - DECLARE @ReviewToFind varchar(1000) = 'Notebooks';
    - SELECT * FROM dbo.tblReviews3
    - WHERE Category = @ReviewToFind
    - As encryption is always done on the client side, you can only use = or IN comparisons; you cannot use >, <, or LIKE.

26. Design and implement Dynamic Data Masking

- Dynamic data masking limits how data is shown in databases.
    - For example, hiding parts of emails or credit cards to users who do not need this information.
    - Data will not be masked for those with Contributor rights or greater, or with elevated permissions (such as the UNMASK or CONTROL permission) on the database.
    - The mask can also be bypassed in Fabric if someone has been shared the warehouse using the ReadAll permission, as they can query the underlying OneLake data.
- The permissions which can be used are:
    - Standard CREATE TABLE and ALTER permissions to incorporate the dynamic data masking when creating a table.
    - The ALTER ANY MASK and ALTER permissions for adding, replacing or removing masks for existing columns.
        - You might want to assign the ALTER ANY MASK for security officers.
        - The ALTER ANY MASK permission is also granted with the CONTROL permission.
    - The SELECT permission is used to view the data.
    - The UNMASK permission allows the columns to be unmasked.
        - This is also granted with the CONTROL permission.
- Permissions can be granted using:
    - GRANT Name_of_permission ON Table_name TO User_name.

- They can also be REVOKEd.
- To mask, then use the following syntaxes:
    - As part of a table definition: Column_Name Column_Type MASKED WITH (FUNCTION = 'name_of_function') NULL
    - As part of an alter column: ALTER TABLE Name_of_table ALTER COLUMN Column_Name MASKED WITH (FUNCTION = 'name_of_function')
- To unmask, use:
    - ALTER TABLE Name_of_table ALTER COLUMN Column_Name DROP MASKED
    - In SQL Server 2022 onwards, you can grant/revoke the UNMASK permission for a database, schema, table, or column to a user, database role, or Microsoft Entra identity/group.
- The functions are:
    - default(). This will show:
        - xxxx (or fewer x's) for strings, 0 for numbers, 1 January 1900 for date/time, and ASCII value 0 for binary, varbinary or image data types.
    - email(). This will show the first letter of the email address, followed by XXX@XXX.com.
    - random(first_value, last_value), to show a random number between first_value and last_value.
        - For example, random(1, 10)
    - partial(prefix, [padding], suffix) to show a random string. prefix and suffix contain the maximum number of first/last characters to be shown.
        - For example, partial(1, "XXXXXX", 2)
    - datetime("__"). This masks what is in the argument. The argument could be:
        - Y, M or D (upper case) for year, month and day,
        - h, m or s (lower case) for hour, minute and second
        - This is available in SQL Server 2022 onwards.
- Note – this technique only masks the data – it does not stop intelligent guesses.

    - For example, let's say Country was masked with the default(), and I was logged in a user which returns masked data.
    - SELECT * FROM tblTable would return xxxx in the Country column.
    - However, while SELECT * FROM tblTable WHERE Country = 'United States' would still return xxxx in the Country column, it's fairly obvious what the actual Country is.
- Therefore, this should be used in conjunction with other security, such as object-, row-, and column-level security.

27. Design and implement Row-Level Security (RLS)

- Row-Level Security (RLS) allows you to control who has access to individual rows in your tables.
    - Available in SQL Server 2016 and onwards.
    - This can be used in conjunction with Object Level Security (OLS - topic 28), data encryption (topic 25) and Dynamic Data Masking (topic 26).
- It can:
    - filter rows that can be used in SELECT, UPDATE and DELETE statements.
    - stop users from writing data which they would no longer be able to read (AFTER INSERT/UPDATE and BEFORE UPDATE/DELETE).
- It is typically used to:
    - allow a department/user to see their own rows, but not other user's rows,
    - while optionally allowing senior managers to see all rows.
- There are three things which are needed to implement RLS:
    - Create a Function. This is used indicate which rows a user should see.
        - This function needs to RETURNS TABLE WITH SCHEMABINDING.
        - SCHEMABINDING is used to prevent any objects which are being referenced from being DROPped or ALTERed.
        - This returns a 1 when the user should see the row, or NULL when they should not.
- You then need to create:
    - CREATE SECURITY POLICY PolicyName

        - ADD FILTER PREDICATE PredicateName(ColumnName) -- (ColumnName) is optional, but is often used.
        - ON TableName
        - WITH (STATE = ON);
        - -- (ColumnName) is optional, but is usually used.
        - To disable the policy, ALTER it with STATE = OFF or DROP it.
- and finally, add SELECT permissions to the function.
- You can check what policies and predicates are currently being used using the system tables:
    - sys.security_policies and
    - sys.security_predicates.

28. Design and implement object-level permissions

- You can:
    - add permissions using the GRANT statement,
    - actively stop access using the DENY statement, and
    - reverse a GRANT or DENY statement by using the REVOKE statement.
- Next, you add the permission:
    - SELECT - read tables and views.
        - You can add in brackets/parentheses columns,
    - DELETE, INSERT, UPDATE - alter data,
    - EXECUTE - run stored procedure,
    - VIEW CHANGE TRACKING - to track changes (see topic 52c),
    - ALTER - change the structure of an object (like a table).
        - Does not allow dropping the table or reading the data.
    - CREATE TABLE/VIEW/PROCEDURE
- Then, you say what you want the permission to be applied ON (the securable). This includes:
    - ON SCHEMA::
    - ON OBJECT::, which includes tables, views, procedures and functions.

- This you say who it is going to be applied TO (the principal). This includes:
    - TO LoginName,
    - TO DatabaseUser,
    - TO ServerRole/DatabaseRole,
        - You can ADD MEMBER into a previously CREATEd ROLE, and then apply permissions to the role, which will cascade to its members.
    - TO ApplicationRole.
- Without a GRANT permission (which has not been DENYed or REVOKEd), principals do not have any access to the securable.
- Note:
    - A table-level DENY currently does not take precedence over a column-level GRANT.
    - This inconsistency in the permissions hierarchy has been preserved for the sake of backward compatibility. It will be removed in a future release.
- In Microsoft Fabric, you can also go to Security - Manage SQL security to:
    - Add users to existing roles
        - Click on the existing role and click on "Manage access" to add additional users.
    - Add custom roles
        - You can then get the role a name, and for each Schema, GRANT Select, Insert, Update, Delete and Execute privileges.

29a. Implement secure database access, including passwordless - Azure SQL Database

- There are two main ways to access an Azure SQL database:
- SQL Server Authentication.
    - This requires a user name and password.
    - The Password is stored in the master database (if the account is linked to a login)
    - Your password needs to be sufficiently complex:
        - Between 8 and 128 characters,

    - Must contain at least 3 of: upper case, lower case, digits and symbols,
    - Does not contain the account name

- Microsoft Entra.
    - This uses Microsoft Entra ID (previously known as Azure Active Directory) to authenticate:
    - Advantages include:
        - More secure than SQL Server authentication,
        - Permission management is generally added into groups, not individuals, reducing complexity,
        - A centralized place for password rotation, monitoring, and conditional access controls (requiring additional things to be in place before access is granted),
        - Can use managed identities for Azure resources (no need for passwords),
        - Allows for multifactor authentication, including text/SMS message and mobile app interactions,
        - Can be used in conjunction with Azure Policies.
    - For users, methods include:
        - Microsoft Entra Integration (Windows Authentication) - extends your Windows login,
        - Microsoft Entra MFA (Multi-factor authentication) - additional security checks,
        - Microsoft Entra Password - uses user credentials in Microsoft Entra ID,
        - Microsoft Entra Default - uses credential caches on your machine.
    - For services or workload identities, methods include:
        - Managed identities - a token-based authentication system, where Azure validates the relationship (see topic 31),
        - Microsoft Entra service principal name and application/client secret - uses passwords; not recommended.
        - Microsoft Entra Default - uses credential caches on your machine.

- The Microsoft Entra admin can create other Microsoft Entra logins and users ("principals"). To do this:
    - In the Azure portal, go to the SQL server - Access control (IAM),
    - Click on Add - "Add role assignment",
    - Search for and click on "Reader" and click on Next,
    - In the Members tab, click on "+ Select members", select the members, and click Select.
    - Click on "Review + assign" twice.
- In the Azure SQL Database, using an admin, and connected using an Microsoft Entra login, enter:
    - CREATE USER [NameOfUser] FROM EXTERNAL PROVIDER;
- If you are using SQL Server Authentication, then, in SSMS:
    - Go to the relevant database
    - Right-hand click on Security, then go to New - User...
    - Select the type of user - usually this will "SQL user with login".
    - Enter the additional details.
- You can then provide access to objects, such as tables (see topic 28).

29b. Implement secure database access, including passwordless - Microsoft Fabric

- Fabric SQL databases, like other objects in Microsoft Fabric, use Microsoft Entra authentication. Users need to either:
    - Be part of a workspace role. This gives them access to all of the objects in a workspace.
        - Viewer and above gives the following permissions:
            - Read - connect to the database,
            - ReadData - read data and metadata, and
            - ReadAll - read mirrored data directly from OneLake files,
        - Contributor and above also gives:
            - Write permission - full administrative access and full data access.

    - Both Members and Admin also have:
        - Share permission - Share the item and manage the item's permissions.
    - To add users to a workspace role:
        - Go to the workspace,
        - Click on "Manage access",
        - Click on "+Add people or groups",
        - Enter the user and the required permission, and click Add.
    - You can also remove users or change their permission in the "Manage access" pane.
- Have the item shared by a Member or Admin.
    - click on the ... next to the SQL database, and go to Share.
    - Enter the people, and optionally click on:
        - Read all data using SQL database,
        - Read all data using SQL analytics endpoint,
        - Read all data using Apache Spark and subscribe to events
        - Build reports on the default dataset,
        - Notify recipients by email, and optionally add a message
    - If the item is shared without "read all data" permissions, then access to data will be to given using SQL permissions (see topic 28).
    - The recipient can then use the links in the email received (if any), or use the OneLake catalog and click on "Open".

30a. Implement auditing - Azure SQL Database

- Azure SQL Database writes database events to an audit log in an Azure service.
    - Retain categories of database actions, which include:
        - Batch completed group
            - When T-SQL have finished running, including stored procedures.

        - Up to 4,000 characters are captured in the statement and data_sensitivity_information per action (any more is excluded).
    - Database Authentication Succeeded
    - RPC completed - a remote procedure call
    - Successful/Failed database authentication
        - A principal has successfully logged into/failed to log into into a contained database
- Report on database activity, using preconfigured reports and a dashboard.
- Analyze reports to find unusual events, activities and trends.

- It allows you to:
    - Help with regulatory compliance and standards (but it does not guarantee them),
    - Understand database activity,
    - Find out about discrepancies/anomalies.
- To set it up:
    - Go to the Azure SQL Database or Server
        - Auditing can be set up for the Database independently to the Server.
    - Go to Security - Auditing.
    - Enable "Enable Azure SQL Auditing".
    - Select the audit log destination:
        - An Azure Storage account,
        - a Log Analytics workspace, to be used by Azure Monitor logs, and/or
        - an Event Hub
        - Note: for Azure SQL Server (not Database), you can also Audit Microsoft's support engineers, if they work on your server.
- You should then configure the destination:

- For Azure Storage accounts:
    - Select the subscription and storage account
        - You can create a new Storage Account here, if needed. The Name should be 3-24 characters, using lower case and numbers only.
        - It is suggested that you use a General-purpose v2 storage account.
    - Select the Storage Authentication Type.
        - Microsoft recommends using Managed Identity, as Storage Access Keys may create a security risk if they have been compromised.
        - By default, the server's primary managed identity will be used.
    - In Advanced properties, you can select the Retention period for the log, in days.
        - The default is zero, which is unlimited.
        - It can go up to around 9 years.
        - If this is changed from zero to another number, then it will only apply to new logs, not existing logs.
- For Log Analytics workspaces:
    - Select the subscription and the existing Log Analytics workspace.
- For Event Hubs:
    - Select the subscription, the existing Event hub namespace, and the Event hub policy name.
    - Optionally, select the event hub name.

- Finally, click Save (at the top of the screen).
- Once it has been saved, and logs have been generated, for Log Analytics and Storage accounts, you can click "View audit logs" (at the top of the screen). Alternatively:
    - For Log Analytics workspaces, you can also open your workspace, go to General - Logs, and use a query to view the audit logs.

- For Event Hubs, you need to set up a stream to consume the events and write them elsewhere.
- For Azure Storage Accounts:
    - In SSMS, you can go to File - Open - Merge Audit Files, Sign in, and select:
        - Tenant
        - Subscription
        - Storage Account, and
        - Blob (Binary Large Object) Container (sqldbauditlogs).
    - It will then be displayed in SSMS, and you can click on any item to get more information.
    - You can use the system function sys.fn_get_audit_file,
    - You can use Power BI,
    - You can download log files from your Azure Storage Account.

30b. Implement auditing - Fabric SQL Database

- To configure auditing in a Fabric SQL database, open it and:
    - go to Security - Manage SQL Auditing.
    - Turn "Save Events to SQL Audit Logs" to on.
    - You can select which event types to record:
        - Permission changes and login attempts - successful and unsuccessful attempts to sign into the database, and/or
        - Data reads and writes - SELECT, INSERT, UPDATE, DELETE and EXECUTE commands, and/or
        - Schema changes - CREATE, ALTER, DROP and TRUNCATE commands, and/or
        - Custom events - you can define which event(s), or
        - Audit everything.
        - You can also use the Advanced - Predicate Expression to filter the database conditions

    - You can also select how long to keep these logs, in terms of the number of 365 days, 30 days and individual days.
        - Using zero for all of them means that logs will be retained until something else deletes them.
    - Then click Save.
- Audit logs can be queries using sys.fn_get_audit_file and sys.fn_get_audit_file_v2
    - The first argument is the location of the audit logs (URL), which comprises of:
        - https://onelake.blob.fabric.microsoft.com/,
        - The Fabric Workspace ID - this is the string in the Fabric database URL after /groups/ and before /sqldatabases/ (it should be followed by a /),
        - The Fabric SQL Database ID - this is the string in the Fabric database URL after /sqldatabases/ and before ?experience
        - /Audit/sqldbauditlogs/
    - The second argument is the Initial File name for the audit logs. If you do not know, use DEFAULT.
    - The third argument is an audit record offset within the second argument's file. You can also use DEFAULT.
- For the _v2 function:
    - The fourth argument is the start time for retrieving the logs - for example: '2028-01-02T03:04:05Z'. You can also use DEFAULT.
    - The fifth argument is the end audit time.
- It is recommended that you use the _v2 function - use the fourth and fifth argument can provide performance improvements.

31. Secure model endpoints, including Managed Identity

- For Microsoft Fabric, see topics 28 and 29.
- In Azure SQL Database, there are two types of Managed Identity:
    - A system-assigned Managed Identity
        - This is created in Microsoft Entra ID,
        - It is linked to the Azure SQL Database, and

    - It is deleted when the Azure SQL Database is deleted.
  - A user-assigned Managed Identity:
    - You can create it and assign it to one or more Azure Resources.
- To create a user-assigned Managed Identity:
  - In the Azure Portal (portal.azure.com):
    - Search for and click on "Managed Identity",
    - Click "+ Create",
    - Select a Subscription, Region group, and Region,
    - Enter a Name,
    - Click "Review + create" and "Create",
    - Once created, click on "Go to resource".
  - In the Azure SQL database:
    - Go to Security - Identity,
    - Click "+ Add",
    - Select the subscription and the Managed identity.
    - Click "Add".
    - Click "Save" (at the top of the window).
- You could then use either Managed Identity when called by models (see topics 58 and 65).

32a and 32b. Secure GraphQL, REST, and MCP endpoints

- You can secure endpoints by:
  - only exposing the tables, queries and stored-procedures that you wish to have exposed.
  - only using the actions that you wish to have exposed:
    - create, read, update and delete for Tables and Views,
    - execute for Stored Procedures
  - only exposing the fields for a particular action. For example:

    { "action": "read",

"fields": {

"include": ["Column1", "Column2"],

"exclude": ["Column3"] }

- only exposing the rows for a particular action, using policies for read, update and delete actions (not for create or execute):

{"action": "read",

"policy": { "database": "@column1 eq 1“ } }

- using the Authenticated or custom roles when users have logged in. Authentication can be dab configure -- in three different ways:
  - Cross-Origin Resource Sharing (CORS). This allows an authorization to come from a different domain. You can specify which domains in --runtime.host.cors.origins
  - A different authentication provider. This can be configured in --runtime.host.authentication.provider,
  - A JSON Web Token (JWT). This can be configured in --runtime.host.authentication. jwt.audience and jwt.issuer.
- disabling “allow-introspection” for the graphql endpoint.
  - It is automatically disabled in production mode.

32c. Secure MCP endpoints

- To help secure your MCP endpoint:
  - It needs to require authentication - do not make it anonymous (even if you are able to).
  - Do not have passwords in plain text - put them into Azure Key Vault.
  - If the MCP server calls elsewhere, it should have the least privilege (permissions) for these calls.
    - Do not give access to additional objects (for example, tables with data, or views) if not needed.
    - Do not give more abilities than needed. If it only needs to read, do not give it right permissions.
  - Always log and monitor tool calls.

Optimize database performance

33. Recommend database configurations

- Azure SQL Database can be:
    - Single database.
    - Elastic pool.
        - Multiple databases on a single server.
    - Compute tier:
        - Provisioned – for regular usage patterns, or multiple databases with elastic pools.
        - Serverless compute – on-off usage with a relative low average compute.
            - Supports automatic pausing and resuming.
            - When the database is paused, you only pay for storage.
- Azure SQL Database allows for:
- vCore purchasing model
    - Specify separate amount of Number of vCores, memory, and amount/speed of storage.
    - Maximum of:
        - 80 vCores at Gen5,
        - 4 Tb memory, and
        - 4 Tb database size (apart from Hyperscale, which has up to 100 Tb).
    - Azure Hybrid Benefit and/or reserved capacity
        - Azure Hybrid Benefit allows you to bring in your existing on-prem licenses to the cloud.
        - Reserved capacity is paying in advance at a discount.
    - Uses local SSDs. Provision in 1 Gb increments.
- When creating an Azure SQL Database, you can select the Compute and storage.
    - Choose from:

- General purpose (scaled compute and storage)
    - For most business workloads. Storage latency of 5-10 ms (about the same as SQL Server on a VM).
- Business critical (high transaction rate and high resiliency) – uses
    - When you need low-latency I/O (1-2 ms) or frequent communications between app and database.
    - Large number of updates, or long running transactions that modify data.
    - Higher resiliency, availability and fast geo-recovery and recovery from failures, and advanced data corruption protection.
    - Free-of-charge secondary read-only replica.
- Hyperscale (on-demand scalable storage) – Only for Azure SQL Database – say 100 Tb+ storage.
    - You cannot subsequently change out of Hyperscale. Cost the same as Azure SQL Database.

- Backup Storage Redundancy
    - Geo-redundant backup storage (default and recommended),
    - Zone and Local Redundancy are cheaper for single region data resiliency.

- DTU (database transaction unit)-based purchasing model
    - For light to heavy database workloads.
    - Offers bundles of maximum number of compute, memory and I/O (reads/writes) resources for each class (cannot separate them).
    - Uses Azure Premium disks. Provision in increments of 250 Gb to 1 Tb, and 256 Gb thereafter.
    - Choose from:
        - Basic (for less demanding workloads)
        - Standard (for typical performance)
        - Premium (for I/O-intensive workloads)

    - Please note: Basic and Standard S0, S1 and S2 have less than 1 vCore, and cannot use "Change data capture".
        - Consider Basic, S0 and S1, where database files are stored in Azure Standard Storage (HDD), for development, testing and infrequently accessed workloads.

- Can change service tier on demand (but not out of Hyperscale).
    - Don’t do it when you have a long job running!
- You can find performance recommendations for your SQL Database by going to Intelligent performance - Performance overview/recommendations. In recommendations, you will see:
    - Action,
    - Recommendation description, and
    - Impact (High, Medium and Low).
    - Your database will need to have activities for at least a day before it can make recommendations - maybe longer.
- You can then:
    - Apply a recommendation,
    - Enable Automatic Tuning to automatically apply:
        - Force Plan - uses a specific, good plan from Query Store
        - Create/Drop Index - to help speed up regularly used queries.
            - Too many indexes could be detrimental to performance.
    - You can also use:
        - ALTER DATABASE [DP-800]
        - SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);
- You can also use ALTER DATABASE SCOPED CONFIGURATION to change configurations at the database level.
- For example, MAXDOP = number
    - For example, ALTER DATABASE SCOPED CONFIGURATION MAXDOP = 8
    - You can see the current number using:

    - SELECT [value] FROM sys.database_scoped_configurations WHERE [name] = 'MAXDOP';

- You can also see/change it in SSMS by:
    - right-hand clicking on the database,
    - going to Properties, and
    - going to the Configurations tab.
- This shows how many Degrees of Parallelism (DOP) can be used - this is the number of processors to use for queries in parallel.
- Since 2020, it is set at 8 by default.
- There is no "best" number, as it depends on workload.
- If you are having performance problems, try experimenting with a different number.
    - Higher figures can speed CPU-intensive queries.
    - Too high a figure can stop other queries from running, leaving to timeouts, errors and outages.
    - For this reason, Microsoft suggests that you avoid MAXDOP 0 (as many as possible - up to the lowest of 64 or the total number of logical processors), as it may cause problems.
    - A figure of 1 would mean that parallel threads are not used. This can significantly increase compute time.
- You can also use it at the individual query level by using OPTION (MAXDOP number) - note: there is no equals sign.
- You can also use it when CREATEing or ALTERing an index.

- Another example is OPTIMIZE_FOR_AD_HOC_WORKLOADS
    - When a query is first run, the plan is stored in the Query Store.
        - The plan shows how the query is to be run. This needs to be calculated if it is not in the Query Store.
        - Storing the plan means that subsequent runs do not need the plan to be calculated.
    - However, multiple queries can take a lot of memory, which is not helpful if the query is not run again.

        - If OPTIMIZE_FOR_AD_HOC_WORKLOADS is turned ON, then a compiled plan stub is stored the first time the query is run.
            - This uses less memory.
        - When it is run a second time, the stub is removed, and a full compiled plan is added.
- If you have an older database, then you might want to check the Compatibility Level.
    - Higher levels equate to more recent version of SQL Server.
    - There may be technical reasons why your database need to be running at a lower compatibility level.
    - To check it:
        - use SELECT * FROM sys.databases, and go to the compatibility_level, or
        - in SSMS:
            - right-hand clicking on the database,
            - going to Properties, and
            - going to the Options tab.
- Generally, you should only change the configuration on production databases if you have a specific reason for it.
- In Microsoft Fabric, if you click on the ... next to a database and go to Settings, you can go to the "Backup retention policy" and adjust the number of days in the retention period.
    - This can be between 1 and 35. The default is 7.

34. Preserve data integrity and consistency by using transaction isolation levels and concurrency controls

- Databases follow the ACID properties:
    - Atomicity - each transaction is a single unit, and commits (succeeds) or fails (rollback) as the unit.
    - Consistency - Data committed needs to be logically correct.
    - Isolation - each transaction should be isolated from each other.

    - Durability - if the system stops working, any uncommitted transactions should be dropped.

- Transactions are isolated from each other using locks.
    - This shows that one transaction is needing to access data.
    - Locks can be for reading or for writing.
    - Locks can be at the row level, the page level (a collection of rows) or the table level.
- Blocking can occur when:
    - Session 1 locks a resource (e.g. row, page or entire table), and then
    - Session 2 requests that resource.
- Blocking is caused by:
    - Poor transactional design, or
    - Long running transactions.
- To emulate this, we are going to have an explicit transaction.
    - Implicit transactions automatically add a BEGIN and COMMIT TRANSACTION.
    - Explicit transactions require you to add the BEGIN, and COMMIT/ROLLBACK TRANSACTION.
    - Transactions are not finished until they reach the COMMIT/ROLLBACK TRANSACTION.
- Session 1
    - BEGIN TRANSACTION
    - UPDATE [SalesLT].[Address]
    - SET City = 'Toronto ON'
    - where City = 'Toronto'
- Session 2
    - BEGIN TRANSACTION
    - UPDATE [SalesLT].[Address]
    - SET City = 'Toronto'

- where City in ('Toronto ON', 'Toronto')

- The lock modes include:
    - S - Shared lock
        - For read operations, like SELECT statements, which do not change/update data,
        - Multiple transactions can have S locks.
        - No other transactions can modify data while S locks exist.
    - U - Update lock.
        - Prepares to update, but are not updating,
        - Only one transaction can have a U lock.
    - X - Exclusive lock
        - Used when data is being modified, such as INSERT, UPDATE, or DELETE.
    - I - Intent hierarchy.
        - These are added at a page or table level to prevent other resources from adding Shared or Exclusive locks to a row or page (at a lower level).
        - They can be combined with other locks - for example, IX.
        - 34. Preserve data integrity and consistency by using transaction isolation levels and concurrency controls
- To reduce blocking, you can change the TRANSACTION ISOLATION LEVEL of a session:
    - SET TRANSACTION ISOLATION LEVEL ...
    - READ UNCOMMITTED – No blocking, but would have dirty reads. This is the default.
        - Dirty reads have uncommitted dependences, when a transaction reads a row which is currently being updated by another transaction.
        - It also allows for non-repeatable reads (inconsistent analysis). This is when another transaction reads the same row multiple times, with different data.

    - It also allows for phantom reads. This is when a second transaction issues the same query, but when a different set of rows are returned.
    - You can use WITH (NOLOCK) or WITH (READUNCOMMITTED) at the end of a table in a query's FROM clause to use this with a single query.

- READ COMMITTED – No dirty reads, as would not read statements that have been modified but not committed.
    - If READ_COMMITTED_SNAPSHOT is OFF (the default on SQL Server), may block.
    - If READ_COMMITTED_SNAPSHOT is ON (the default on Azure SQL Database), uses a snapshot and therefore does not block.
    - Read Committed does not allow for dirty reads, but does allow for non-repeatable reads and phantom reads.
    - You can use WITH (READCOMMITTED) or (READCOMMITTEDLOCK) at the end of a table to use this with a single query.
- REPEATABLE READ – No dirty reads and no non-repeatable reads, but blocks. It also allows for phantom reads.
    - You can use WITH (HOLDLOCK) at the end of a query to use this with a single query.
- SNAPSHOT – The data read remains the same until the end of the transaction. No blocks unless the database is being recovered.
    - No dirty reads, no non-repeatable reads, and no phantom reads.
    - Only Schema stability (Sch-S) locks are acquired.
    - Needs ALLOW_SNAPSHOT_ISOLATION to be ON.
    - You cannot run it for just one query, unless it is memory-optimized - then you can use WITH (SNAPSHOT)
- SERIALIZABLE - No dirty reads, as would not read statements that have been modified but not committed. However, blocks updates/inserts.
    - Transactions are completely isolated from each other.
    - Read and write locks are kept on selected data until the transaction ends.

    - You can use WITH (SERIALIZABLE) in a query's FROM clause to use this with a single query.

- To see the current level, use
    - DBCC USEROPTIONS
- There are database options as well.
    - ALTER DATABASE NameOfDatabase SET ALLOW_SNAPSHOT_ISOLATION ON
        - DML statements start generating row versions – allows snapshots but doesn't enable it.
    - ALTER DATABASE NameOfDatabase SET READ_COMMITTED_SNAPSHOT ON
        - DML statements start generating row versions – means that TRANSACTION ISOLATION LEVEL READ COMMITTED does not block.
- Preserve data integrity and consistency by using transaction isolation levels and concurrency controls

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read uncommitted	**Yes**	**Yes**	**Yes**
Read committed	x	**Yes**	**Yes**
Repeatable read	x	x	**Yes**
Snapshot	x	x	x
Serializable	x	x	x

35a. Evaluate query performance by using query execution plans

- Execution plans are created when a query has been run, to identify an efficient method of achieving the query results.

- The Query Optimizer builds at least one query execution plans (also known as query plans, or execution plans).
- It then evaluates those plans to work out the best way, considering compilation time and how optimal the plan is. For example:
    - Which order the source tables are accessed,
    - How the data is obtained from the tables, and
    - How computations, filtering, aggregating, and sorting are done.
- There are two different types of query execution plans:
    - Estimated - this is created before the query is run,
        - In SSMS, to view this, click on the "Display Estimated Execution Plan" (to the right of "Execute"). This creates the plan immediately without running the query.
        - Alternatively, run SET SHOWPLAN_XML ON. This will produce estimated execution plans in XML form (which you can then click on) until you switch it to OFF.
        - It may show missing indexes.
            - To show the CREATE statement for missing indexes in a separate window, right-hand click on the plan and go to "Missing Index Details".
        - You can right-hand click on a plan for more options, including:
            - Analyze Actual Execution Plan,
            - Zoom In/Out, Custom Zoom, Zoom to Fit.
    - Actual - this is after the query has been run.
        - You can click the "Include Actual Execution Plan" button. This does not run the query or create the plan, but when the query is separately run, the plan will be created.
        - Alternatively, run SET STATISTICS XML ON/OFF.
    - Additionally, Live query statistics are the actual execution plan during a query's run (used for longer queries).
- You can compare two different queries' plans by running them in the same batch.
    - Note the "Query cost (relative to the batch)". If one query is a bigger percentage than the other, then you may wish to investigate why.

- In the plan, the thickness of the arrow indicates how many rows have moved from one operator to another.
    - You can click on an arrow or an operator for more details.
- Underneath each operator you can see the percentage cost needed for that operator.
- Looking at operators:
    - "Scans" means that each row of the table is being assessed separately.
        - Where this time is non-trivial, an index should be used to convert this scan to a seek.
    - "Clustered" means that there is a primary key.
    - "Table scan" means that there is no index.
        - This may indicate a lack of indexes that would help.
    - Sorting and aggregating is expensive.
- Joins
    - "Nested loops" happen when one input is small and the other is large and indexed on its join columns.
    - "Merge joins" happen when the two join inputs are not small but are sorted (indexed) on their join column.
    - "Hash joins" are used for large, unsorted and non-indexed inputs.
- SQL Server 2017 and onwards (compatibility level 140) allow for Adaptive Joins.
    - This allows the choice of Hash join or Nested join to be made after the first input has been scanned.
- You can check it is enabled by:
    - Right-hand clicking on the database in SSMS,
    - Go to Properties - Configurations tab.
    - Look at "BATCH_MODE_ADAPATIVE_JOINS".
- To enable it in SQL Server 2019 and later, use:
    - ALTER DATABASE SCOPED CONFIGURATION SET BATCH_MODE_ADAPTIVE_JOINS = ON;
- You can also disable it using a Hint.

        - SELECT ...
        - OPTION (USE HINT('DISABLE_BATCH_MODE_ADAPTIVE_JOINS'));
- If it is making the "wrong" decisions, it could be that the Statistics are out of date.
    - Statistics is information about the distribution of data in a table.
    - You may disable it if you have permissions issues on tables which have huge amounts of data being added/deleted.
        - To turn it off, use ALTER DATABASE NameOfDatabase SET AUTO_UPDATE_STATISTICS OFF WITH NO_WAIT. This is not recommended.
    - The AUTO_UPDATE_STATISTICS_ASYNC determines whether static updates are done asynchronously to query running.
        - If it is synchronous, queries will not run until statistics are updated.
        - If it is asynchronous then queries can run with out-of-date statistics.
    - To update statistics, use UPDATE STATISTICS TableName.
        - You can optionally add the index in brackets/parentheses.

35b. Evaluate query performance by using dynamic management views (DMVs)

- Dynamics management views (DMVs) can be used to look at performance or find out where problems are.
- They include:
    - sys.dm_db_resource_stats, which gets data every 15 seconds and maintains it for an hour. This includes:
        - avg_cpu_percent - the compute average divided by the service tier limit. If this is consistently too high, then you might need to increase the service tier limit.
            - If it is over 80% for a long time, you might want to investigate further.
        - avg_data_io_percent - the data input and output percentage,
        - avg_log_write_percent - transaction log writes percentage,

    - avg_memory_usage_percent - memory use percentage,
    - xtp_storage_percent - in memory storage use percentage,
    - max_worker_percent - maximum simultaneous requests (or workers) percentage,
    - max_session_percent - maximum simultaneous sessions percentage,
    - For DTU-based models (Basic, Standard and Premium):
        - dtu_limit - the current maximum database DTUs,
        - cpu_limit - the number of vCores
    - avg_instance_cpu_percent - average CPU use divided by the instance limit, and
    - avg_instance_memory_percent - average memory use, including user and internal workloads,
    - used_ and allocated_storage_mb.
    - You can use aggregations such as AVG or MAX with these columns.
- In the master database, the sys.resource_stats view includes:
    - start_time and end_time - there is one row for every 5 minutes, and historical data is retained for around 14 days.
    - database_name - the user database,
    - sku - either Basic, Standard, Premium, General Purpose or Business Critical, and
    - other columns from the sys.dm_db_resource_stats view. If the database is part of an elastic pool, the percentages is out of the elastic pool configuration.
- The sys.dm_elastic_pool_resource_stats (every 15 seconds for an hour) and sys.elastic_pool_resource_stats (master database - every 15 seconds for the last 14 days) provides similar information for all elastic pools on the logical server.
- The sys.dm_exec_sessions view shows the current active sessions, including:
    - session_id (see the tab of the session),
    - login_time,

    - host_name - the client workstation,
    - program_name - the client program,
    - login_name - the SQL Server login name which the session is executing under,
        - There is also original_login_name, which is the SQL Server login name which created the session
    - status - either Running (at least one request), Sleeping (no requests), Dormant (session was reset) or Preconnect (in the Resource Governor classifier),
    - last_request_start_time and end_time,
    - reads and logical_reads
    - is_user_process - 0 for a system process, and 1 otherwise,
    - various settings for the sessions, from text_size to deadlock_priority. This includes transaction_isolation_level, which is:
        - 0 - Unspecified,
        - 1 = ReadUncommited,
        - 2 = ReadCommitted,
        - 3 = RepeatableRead,
        - 4 = Serializable, or
        - 5 = Snapshot.
    - row_count - the number of rows returned in the current session
    - database_id - as used in sys.databases. You can retrieve the name using DB_NAME,
    - open_transaction_count - the number of open transactions.
- The sys.dm_exec_requests view shows information about each request in SQL Server. It includes:
    - session_id (see the tab of the session),
    - request_id,
    - start_time,

- status - the request status. This is either background, rollback, running, runnable, sleeping or suspended.
- command - these include SELECT, INSERT, UPDATE, DELETE, BACKUP LOG, BACKUP DATABASE, DBCC or FOR.
- sql_handle. This identifies the batch/stored procedure.
    - To see the SQL text, you can CROSS APPLY it with the sys.dm_exec_sql_text(req.sql_handle)
- cpu_time
    - You could order it by cpu_time desc to see the longest running queries,
- various settings for the requests

35c. Evaluate query performance by using Query Store

- The Query Store:
    - gives you information about query plan choice and performance,
    - allows you to find performance differences between different query plan changes,
    - retains a history of queries, plans and statistics.
- It allows you to:
    - Find and correct plan performance regression,
    - Find the number of times a query was run in a particular time period,
    - Find the top x queries in the top y hours, based on execution time, memory consumption, or waiting on resources,
    - Audit query plans for a particular query,
    - Analyze compute, Input/Output and Memory usage patterns,
- It contains:
    - a plan store for execution plans,
    - a runtime stats store for execution statistics, and
    - a wait stats store for wait statistics data.
- By default, Query Store is:
    - enabled for new Azure SQL Database and Managed Instance databases,

    - not enabled for SQL Server 2016, 2017 or 2019,
    - enabled for SQL Server 2022 and onwards, and
    - not enabled for new Azure Synapse Analytics databases.
- To enable it, use:
        - ALTER DATABASE DatabaseName
        - SET QUERY_STORE = ON (OPERATION_MODE = READ_WRITE) or READ_ONLY
- To change options:
    - Either:
        - right-hand click on the database in SSMS, and
        - go to the Query Store page,
    - or query sys.database_query_store_options
- Options that you can add in the brackets/parentheses include:
    - WAIT_STATS_CAPTURE_MODE = ON. This allows you to capture query wait statistics.
    - MAX_PLANS_PER_QUERY = 300. This is the maximum number of plans maintained for each query. 0 means that there is no limitation.
- To use the Query Store, right-hand click on Query Store for a database in the Object Explorer, and you can select:
    - View Regressed Queries - this shows queries which have got different execution plans.
        - Useful for confirmation about performance problems with individual queries.
        - You can use the filters at the top to sort by different metrics.
        - You can click on a plan to see information about it.
        - You can click on "Track the select query" to track it.
        - You can click on "plan summary in a grid/chart format" (top-right hand corner).
            - A circle means that the query was completed,
            - A square means that the client stored it running.

        - A triangle means that an error stopped it running.
        - The size shows the number of times it was run.
    - To request a plan to be used for a particular query, click on "Force Plan".
- View Overall Resource Consumption, showing duration, execution count, CPU time and Logical reads for each day over the last month
    - Use to identify resource patterns at different times of the day/week.
- View Top Resource Consuming Queries
    - Use to find the queries which have the biggest effect on resources.
- View Queries with Forced Plans
- View Queries with High Variation
    - Use to find queries which have got different performance
- Query Wait Statistics
- View Tracked Queries

- You can also use catalog views, including:
    - sys.query_store_plan - information about the execution plans. This includes query_id, which links to
    - sys.query_store_query - information about the queries. This includes query_text_id, which links to
    - sys.query_store_query_text - this includes the SQL text of the query.
    - sys.query_store_wait_stats - wait information, which includes plan_id (as does sys.query_store_plan)

35d. Evaluate query performance by using Query Performance Insight

- You can also query performance by using Query Performance Insight.
- In the Azure Portal:
    - go to the relevant database, and
    - go to Intelligent performance - Query performance insight.
- In the "Resource consuming queries" tab, you can see:

- The top 5 queries by CPU, Data IO, Log IO, Duration or Execution Count. You can click on customize to go to the Custom tab to change the:
    - Metric type,
    - Time period,
    - Number of queries,
    - Query aggregation (sum, max or avg) - this affects the top graph, and
    - Metrics aggregation (avg, min or max) - this affects the bottom graph.
- The different queries are coloured in the graph as per the table below, which shows:
    - CPU percentage,
    - Data Input/Output percentage, and
    - Log Input/Output percentage.
- You can click on a query to see more information, which includes:
    - A script to get the query text,
    - The CPU, Data IO, Log IO, Duration and Execution by time.

- You can also click on the "Long running queries" tab to see the top 5 queries by Duration or Execution count.
- You can also click on "Recommendation" at the top to see any recommendations, and the past tuning history.

36. Identify and resolve query performance issues, including blocking and deadlocks

- To view locks:
    - SELECT * FROM sys.dm_tran_locks
- To view blocking:
    - SELECT session_id, blocking_session_id,
    - start_time, status, command,
    - DB_NAME(database_id) as [database],
    - wait_type, wait_resource, wait_time,

    - open_transaction_count
    - FROM sys.dm_exec_requests
    - WHERE blocking_session_id > 0
- For the session_id, look at the numbers in brackets at the top of SSMS.
- See also topics 34 and 35.

Implement CI/CD by using SQL Database Projects

37. Design and implement a testing strategy, including unit tests and integration tests

- To create unit and integration tests, you can use SSDT (SQL Server Data Tools) in Visual Studio.
    - You may also be able to use the MSSQL Extension in Visual Studio Code, but as of the time of writing, it was not working properly.
- In Visual Studio:
    - Go to File - New - Project
    - Select the "Query Language - SQL Server Database Project" template. If you can't see it, then:
        - click on "Install more tools and features"
        - in "Other Toolsets", click on "Data storage and processing".
            - You should see the "SQL Server Data Tools" feature be checked.
        - Click "Modify".
    - Enter a Project Name, Location, and optionally check "Place solution and project in the same directory". Then click Create.
    - Right-hand click on the Project in the Solution Explorer and go to Properties:
        - Change the Target Platform to "Microsoft Azure SQL Database".
    - Open View - SQL Server Object Explorer.
    - In the SQL Server Object Explorer pane, click on "+ Add". Then:
        - Click on Azure,

37. Design and implement a testing strategy, including unit tests and integration tests
    - Enter the Server Name (from the Azure SQL Database),
    - Change the Authentication to SQL Server Authentication, and add the user name and password.
    - Click Connect.

- Go to the relevant Projects at the bottom of the SQL Server Object Explorer.
- Create a new query by right-hand clicking on the database and go to "New Query...".
- Write a Stored Procedure that returns a value that can be checked - for example:

```
CREATE PROCEDURE dbo.procTest
@Description nvarchar(200)
AS
BEGIN
    DECLARE @IDNumber INT;
    INSERT INTO dbo.tblTest(Id, Description)
    VALUES (-555, @Description);
    SELECT @IDNumber = ID
    FROM dbo.tblTest
    WHERE Description = @Description;
    RETURN @IDNumber;
END
GO
```

- Go to Build - Build Solution.
- Right-hand click on the database, and go to Publish Data-tier Application.
    - Select the .dacpac file in the bin/Debug folder.
    - Click the Publish button.
        - If you have having problems, saying that the "project which specifies SQL Server 2025 or Azure SQL Database Managed Instance", then got to Build - Clean Solution, then Build - Build Solution.

37. Design and implement a testing strategy, including unit tests and integration tests

- In the SQL Database Project, right-hand click on a Stored Procedure, and select "Create Unit Tests".
    - Fill in the details (using the C# language), and click OK.
- In the next dialog, you will need to click "Select Connection...", select the database connection, and click Select, then OK.
- Enter the Unit Test - for example:

  ```
  DECLARE @RC AS INT, @Description AS NVARCHAR (200);

  DELETE FROM dbo.tblTest WHERE Id = -555;

  SELECT @RC = 0, @Description = 'John Smith';

  EXECUTE @RC = [dbo].[procTest] @Description;

  SELECT @RC AS RC;
  ```

- You can also add a pre-script and post-script, to:
    - Do a clean-up of previous data before running the test, and
    - Do a clean-up of any data you added during the test.
- Delete the existing test condition (which is there to show that there is not an existing test condition).
- Add a Scalar Value Test Condition.
- Change the Properties to:
    - Null expected: False
    - Expected value: -555.
- Open app.config of the Solution Explorer.
    - If you cannot see it, then click "Show All Files".
    - Make sure that the ExecutionContext and PrivilegedContext have got:
        - Initial Catalog=NameOfDatabase;
        - User ID=NameOfUser;
        - Password=Password;
- Go to View - Test Explorer.
    - Go to the test and click on Run.
    - If the test is successful:

    - It should say "Validating ConditionName".,
    - It should have no error messages,
    - It will not say "Is Validated" - just "Validating".

- You can also create tests which run in the pipeline (integration tests).
- These could be designed (for example) in C#, and will query the database and check that the received results are what it expects.
- You would add these to a pipeline - for example:
    - - name: Integration Tests
    - run: dotnet test IntegrationTestName.csproj

38. Create and manage reference/static data in source control

- Reference or static data is not included as part of a SQL Database Projects.
- Generally, you would add static data using a post-deployment script.
    - This would occur after the creation of any objects, including tables.
- To add the data, you could:
    - DELETE any static data and then INSERT it, or
    - MERGE the data
- To create a post-deployment script in Visual Studio Code:
    - Right-hand click on the Project,
    - Click on "Add Post-Deployment Script",
    - Enter a name for the Post-Deployment Script, and
    - Enter your SQL code - for example:

      DELETE FROM dbo.tblCustomers

      WHERE Id IN (-1, -2);

      INSERT INTO dbo.tblCustomers(Id, Description)

      VALUES (-1, 'First name'), (-2, 'Second name');

        - or

      MERGE INTO dbo.tblCustomers as target


```
USING (VALUES (-1, 'First name'), (-2, 'Second name')) AS source (Id, Description)

ON target.Id = source.Id

WHEN NOT MATCHED THEN

INSERT (Id, Description)

VALUES (source.Id, source.Description);
```

39. Create, build, and validate database models by using SQL Database Projects, including SDK-style models

- A SQL Database Project is a local representation of SQL objects such as tables, stored procedures or functions.
- It allows to be integrated into a CI/CD (continuous integration and continuous deployment) workflow, which allows for code to be build, tested and deployed.
    - It can also be used with object-relational mappers (ORM) - for example, EF Core.
    - The result, a build artifact with the extension ".dacpac" (a Data-tier application), can also include a pre- and post-deployment script.
- There are two different types:
    - The original-style (SSDT - SQL Server Data Tools) projects, and
    - The project SDK for SQL projects. This is recommended for new development. In addition, to the functionality of the original-style projects, it also allows for:
        - Cross-platform .NET 8+ support,
        - NuGet package references for database references, and
        - Default globbing pattern (matching files using wildcards) for project .sql files.
    - Original style SQL projects can be converted into SDK-style projects.
    - It is used by SQL Database Project in Visual Studio Code, and is in preview for Visual Studio 2022+.
    - You've also got a lot of functionality in SQL Server Management Studio (SSMS), except for Opening original-style projects and Publish options/properties.

- In Visual Studio Code:
    - Install the "SQL Database Projects" extension:
        - Go to Terminal - New Terminal,
        - Ensure that the .NET SDK is downloaded and installed.
            - To test that it is installed, run dotnet --list-sdks
            - To download it, go to https://dotnet.microsoft.com/en-us/download/dotnet
        - Go to Extensions (on the left-hand side).
        - Search for "SQL Database Projects", and then click on "Install".
        - Once it is installed, you should see the "Database Projects" and "SQL Server" icons on the left-hand side.
        - It supports SQL Server, Azure SQL, Azure Synapse Analytics, and Fabric's SQL database.
- To create a new project:
    - Go to "Database projects" (on the left-hand side), and click "Create new".
    - You are then asked what type of project template to use:
        - Azure SQL Database, and
        - SQL Server Database.
    - Then enter your project name, the Project Location, and the SQL Server version, including:
        - Azure SQL Database,
        - SQL Server (choose between 2012 and the latest version),
        - Azure Synapse SQL Pool (or Serverless SQL Pool),
        - Synapse Data Warehouse in Microsoft Fabric, and
        - SQL Database in Fabric.
    - Then say whether you want to configure SQL Project Build as the default build for this folder.
- Then you can add objects to the project by right-hand clicking on the database project and going to:

    - Add Item, Existing Item or Folder,
    - Add Table, View, Stored Procedure.
- You can then edit the definition.
- Prior to building the project, you can adjust the Code Analysis settings.
    - Right-hand click on the project and go to Code Analysis.
    - There you will see around 16 different types of rules: design, naming or performance problems.
    - You can change what will happen if those rules are violated:
        - Disabled (Nothing),
        - Warning, or
        - Error.
    - You can also check "Enable Code Analysis on Build" and then click OK.
- To build and validate the project, right-hand click on the project and go to Build (you could also go to "Build with Code Analysis").
    - The .dacpac file will be stored in the build output (the default location is bin/Debug/<project_name>.dacpac).
        - You can right-hand click on the Project and go to "Open Containing Folder", then go to the bin folder, then the Debug folder.
    - Once it has been built, you can press a key to close it.
- When the project is built:
    - Relationships are validated (the target table/columns need to exist in the project),
    - The .sqlproj file includes the "target platform" (for example, SQL Server 2025), and you cannot use functionality which is more recent that that target.
    - It builds a .dacpac file in the bin/Debug folder.
        - This file can be used on multiple databases, if you wish.
- It can then be deployed by tools, including SqlPackage.
    - It uses the relationships to create each object in the correct order (so a view will not be based on a table/columns which currently do not exist).

- It works out the steps needed between the .dacpac and the target database and only does those steps which need to be done.
    - For example, instead of dropping and re-creating a table to add an additional column, it just adds the additional column.

40. Configure source control for SQL Database Projects

- To use source control in Visual Studio Code, you need a version of source control. We will be using "Git" in this course. Git is a Distributed Version Control System, or DVCS.
    - Distributed:
        - users do not require connection to a centralized version (though they can use a centralized version).
        - Instead, they can have their own copy.
    - Version control system: It tracks the files' history:
        - Who, what, why and when were changes made.
        - You can also add comments and issues.
- It allows multiple people to work on a single project, including internationally.
- You can restore earlier versions if needed.
- Users can either work from a centralized version, or their own version.
- To use source control in Visual Studio Code, first of all you need to download Git from https://git-scm.com/install/
    - To check whether you already have Git installed, in a Visual Studio Code terminal enter: git --version
    - It is available for Windows, Mac and Linux/Unix.
    - The latest version of the Windows download requires a 64-bit version.
        - The last full version of the Windows download which can be installed on a 32-bit computer is version 2.48.1 from February 2025.
        - This is because of lack of support of one of its components.
- Then, in Visual Studio Code, go to Terminal - New Terminal, and enter the following to establish your identity:
    - git config --global user.name "Your Name"

    - git config --global user.email "your.email@example.com"

- You can then go to Source Control (on the left-hand side) and:
    - click on "Initialize Repository" to create a new Git repository, and then
    - click on "Commit", then "Yes", then add a commit message into line 1 of the new window (for example, "First commit"), then "Commit", then "Save".
    - These initial files have been added to your local Git repository (source control).
- Once you have a GitHub account, you can then go back to Visual Studio Code - Source Control (on the left-hand side), and click on "Publish Branch".
    - You may need to click on "Sign In" (at the top of the screen) - Continue with GitHub first.
        - A web browser may appear, asking you to authorize Visual Studio Code with GitHub. If so, click "Continue", then "Authorize Visual-Studio-Code".
    - If you need to sign into a different GitHub account, you can do that by:
        - going to the Person icon (at the bottom-left of the window),
        - clicking on the GitHub account, and
        - going to Sign Out.
- It will then invite you to create a new repository, which will store your files in GitHub.
    - You should choose whether it is public or private.
        - Public means that everyone can see the repository.
        - Private means that you can choose who can see the repository.
        - Regardless, you can choose who can commit.
    - It then will upload the files to a new repository.
        - You can click on "See on GitHub" to see your repository online.
    - It may ask you: "Would you like Visual Studio Code to periodically run "git fetch"? If you say "yes", then Visual Studio Code will periodically retrieve information from GitHub

- If someone else changes/adds/removes files on your repository, Visual Studio Code will retrieve those changes.

41. Manage branching, pull requests, and conflict resolution

- The default branch is called "main".
- You should do any development changes in a separate branch, and then pull it into the "main" branch.
- To create a new branch, in the Code tab of the repository:
  - Click on the "main" drop-down (this shows the current branch),
  - Click on "View all branches",
  - Click on "New branch",
  - Give the branch a name, and click on "Create new branch".
- You can then make any changes in this branch.
- Once you have created the changes, you then create a Pull Request to incorporate them into the "main" branch. In your repository:
  - Go to Pull Requests (at the topic).
  - Click on "Compare & pull request".
  - Click on "Create pull request", then "Merge pull request", then "Confirm merge".
- It might be that there are conflicting changes to more than one branch which need to be resolved.
- To look at the conflict:
  - Go back to the Pull requests tab.
  - Click "New pull request".
  - Change the "compare" branch.
  - Click "Create pull request".
    - You will see "Can't automatically merge. Don't worry, you can still create the pull request".
  - Click "Create pull request" again.
  - Then click "Resolve conflicts" - Edit on the web.
  - The changes will be shown between <<<<< ====== >>>>>. You can:

        - Manually edit them, or
        - click on Accept current/incoming/both changes.
    - And then click "Mark as resolved", then "Commit merge", then "Merge pull request", then "Confirm merge".

42. Implement secrets management

- Because your workflow is available in plain text in your repository (in the folder ".github/workflow"), the connection string, including any passwords, are available in plain text as well.
    - You should not add, commit or push any sensitive information, including personal data, or login information.
- To change this, you can add a secret, and then reference that secret in the workflow.
- To add a secret, in the Repository:
    - Go to Settings (at the top), then Secrets and variables - Actions (on the left-hand side).
    - Click on "New repository secret".
    - It needs a name (for example, SQLCONNECTIONSTRING), and then the secret itself (the connection string).
    - Then click "Add secret".
- You can then edit the workflow:
    - In the Repository, go to the Code tab, then the ".github/workflows" folder and click on the yml file.
    - Then click on the pencil icon to edit it.
    - Change the connection-string to:
        - connection-string: ${{ secrets.SQLConnectionString }}
    - Then click on "Commit changes...".
    - You can then see if it works by going to the Actions tab (at the top).

43. Detect schema drift by using SQL Database Projects

- Schema drift occurs when changes happen outside of the SQL Database Project:
    - A column or object is added in the database itself, but not in the SQL Database Project.

    - A column or object is added in the database itself, and a conflicting column/object is added into the SQL Database Project.

- To check for schema drift:
    - Make sure that you have saved and built the latest SQL Database Project.
        - Make sure that the Build has been successful.
        - Do not Publish it, otherwise all of the changes will be made.
    - Right-hand click on the SQL Database Project and select (Build first, and then "Schema Compare").
    - You will see your existing SQL Database Project in the "Source" section.
    - Click on the ... in the Target section, and you can select:
        - A Connected Database (and then select the Server and Database)
        - Data-tier Application File (.dacpac), or
        - Another SQL Database Project.
    - Click "OK" to close this dialog box.
    - You can optionally click on "Options" (at the top), to:
        - View/change the rules for comparison, and
        - View/change which Object Types to include.
    - You can also optionally click on "Switch Direction", to swap the Source and Target.
    - Then click "Compare".
        - This may take a few minutes.
- You will then see the results of the comparison in a grid.
- You can see the differences ordered by Action (for example, Delete first, then Change, then Add).
- You can click on an object to see the differences (+s and -s).
- You can then click on "Generate Script" to create T-SQL commands that would alter the Target to match the source.
    - It may be that a table is not completely dropped and recreated, but altered, for example.

44. Update an SQL database project and deploy changes

- Once a project has been built, you can Publish it.
    - First, you should create a Connection.
        - Go to SQL Server on the left-hand side,
        - Click "+ Add" next to Connections,
        - Enter a Profile Name.
        - Enter the Server name and authentication details,
        - If you do not need the server certificate to be validated, then check "Trust server certificate",
        - Enter the database name,
        - You can change the Encrypt settings, and click on Advanced if needed.
        - Once finished, click "Connect".
    - Next, you should create a Publish Profile.
        - Right-hand click on the Project and go to Publish Profile.
        - Enter the name of your Publish Profile.
        - You can then edit the profile. It includes:
            - AllowIncompatiblePlatform - this attempts deployment even if there are differences in the platform,
            - BlockOnPossibleDataLoss - stops deployment if there is the possibility of data loss.
            - DropObjectsNotInSource - removes objects in the target which do not exist in the source.
            - IgnoreColumnOrder - Ignores the differences in source and target column orders.
        - If you do edit the profile, then you should save the profile.
    - Then you can Publish:
        - Right-hand click on the Project and go to Publish.
        - You can choose:
            - An existing SQL Server, or

    - A new local Docker SQL Server (running on your computer)

- Then select a Publish Profile, and add the Connection.
- You can then click Publish, or click on "Generate sqlpackage command" if you want to use the command line.
- The dacpac will then be deploy - it may take a few minutes (see bottom-right of Visual Studio Code window).
- Once the SQL Database Project has deployed, then refresh the tables in your Azure SQL Database to see any changes.

- Alternatively, if you right-hand click on the Project, you can also go to "Update Project from Database", to update the SQL Database Project, not the Database.
    - This can be useful if you have an existing database, and want to create an SQL Database Project from it.
    - You can also do that by right-hand clicking on an SQL Server Database Connection (on the left-hand side) and going to SQL Database Projects - Create/Update Project from Database.

45. Create deployment pipeline

- You can automate the building of the SQL Database Project by creating an Action in GitHub:
    - GitHub actions are:
        - free for public repositories that use standard GitHub-hosted runners,
        - free for self-hosted runners,
        - Private repositories have a quota of free minutes of several thousand minutes per month.
            - Once your free quota is used, if you have not previously added a payment method, your usage will be blocked.
    - In the GitHub repository, go to Actions, and in "Simple workflow" click Configure.
    - All of the lines which start with a hash/pound sign are comments - they can be removed.
    - Firstly, give the Workflow a name.
        - It will end with .yml (Yet Another Markup Language).

- Note that it will be saved in the .github/workflows folder.

- Then give the action a name
- Select when the workflow will run - for example, when there is a push/pull request on main, or manually from the Actions tab.
- Then build up the job. The following actions are fine:

```
jobs:

build:

runs-on: ubuntu-latest

steps:

-uses: actions/checkout@v4
```

    - Then add an action to set up .NET:

```
 - name: Setup .NET

   uses: actions/setup-dotnet@v4

   with:

    dotnet-version: '8.0.x'
```

  - (Note - the spacing is important, and with is expecting a key/value pair, which must be indented on the next line)

- Then add an action to Build the SQL Database Project. Use the name from your existing sqlproj file.

```
 - name: Build

   run: dotnet build <<NameOfProject>>.sqlproj
```

  - Make sure you include any folder name as part of the NameOfProject.

- Then click on "Commit changes".
- Afterwards:
  - you can view the workflow run.
  - you can also pause the workflow (stop it from running in the future) by clicking on the ... in the workflow and selecting "Disable workflow".

- If you want to edit the workflow, in your repository go to the .github/workflows folder.

- At the moment, the DACPAC is saved into a temporary location. You can then deploy the package using sqlpackage.
- You can get the "connection-string" from the Azure Portal - the relevant SQL database - Settings - Connection strings, and go to "ADO.NET (SQL authentication)".

```
- name: Deploy
  uses: azure/sql-action@v2
  with:
    connection-string: ${{ secrets.SQL_CONNECTION_STRING }}
    action: 'publish'
    path: 'bin/Debug/MyDatabaseProject.dacpac'
```

- Again, you may need a folder path before the "bin/Debug".

45a. Deployment pipeline - Branching policies

- You can protect a branch or multiple branches to stop unwanted things from happening to it.
- To do this:
  - Click on the Settings (Wheel) icon in a repository.
  - Either:
    - Go to Branches – Add branch ruleset, or
    - Go to Rules – Rulesets – New ruleset – New branch ruleset.
  - Enter a Ruleset Name.
  - The default Enforcement status is “Disabled”, so it will not be enforced.
    - You can change it to “Active”, so that it is enforced.
  - You can optionally create a Bypass list, so that they will not apply to roles, teams or apps.
  - You should then select the Target branches by clicking on “Add target”, and selecting:
    - Include default branch,

    - Include all branches,
    - Include/exclude by pattern – for example:
        - *dev* will target all branches which include “dev”. * means any number of characters, but not including /s.
        - Prod/* will target all branches with prod, then /, then any characters (but not any additional / characters).
    - If you have multiple rules which affect the same branches:
        - the rules which mention the specific branch will have priority,
        - If there is still a tie, the one created first will have priority.

- Then you can set up the branch protection rules. They are:
    - Restrict creations/updates/deletions
        - Only users with bypass permission will be allowed to do this for the corresponding branches.
    - Require linear history.
        - This stops collaborators pushing merge commits; they must use squash merge or rebase merges instead. Note: the repository must allow these for this option to be used.
    - Require deployments to succeed before merging
        - For example, changes must be successfully deployed to a staging environment before merging to the default branch.
        - You can specify which environments can be used.
    - Require signed commits
        - This requires electronic signatures to be used
    - Require a pull request before merging
        - See topic 45b.
        - The following options are available:
            - Requires a number of approvals to be done before merging.
            - Dismiss stale Pull Request approvals when new commits are pushed

        - Require review from Code Owners
        - Require approval of the most recent reviewable push, by someone other than the person who pushed it
        - Require conversation resolution before merging
        - Request pull request review from Copilot.
        - Restrict merging methods to merge and/or squash and/or rebase.
    - Require status checks to pass. You can:
        - Require branches to be up to date before merging,
        - Do not require status checks on creation
    - Block force pushes
    - Require code scanning results
        - From external tools such as CodeQL.
- Once the rules have been set, press “Create”.
- Following creation, you can click on the … next to the rule and either export or delete the ruleset.
    - To edit it, press the ruleset.

45b. Deployment pipeline - Triggers in approvals

- When you are creating a pull request, you can add up to 15 reviewers.
    - These are people who should look at the code and then comment, approve, or request additional changes.
- One of the branch protection rules is:
    - Require a pull request before merging
        - The following options are available:
            - Requires a number of approvals to be done before merging.
            - Dismiss stale Pull Request approvals when new commits are pushed
            - Require review from Code Owners (see topic 45d)
            - Require approval of the most recent reviewable push, by someone other than the person who pushed it

- Require conversation resolution before merging
- Request pull request review from Copilot.
- Restrict merging methods to merge and/or squash and/or rebase.

45c. Deployment pipeline - Authentication tables

- Authentication tables are things which store user names and passwords.
- This means that pipelines are run by users (or managed identities) which have the appropriate level of permissions.
- GitHub itself does not use authentication tables. However, it uses permission levels.
- For organizations, there are predefined roles, including:
  - CI/CD admin - this grants admin access for Action policies, secrets, variable, usage metrics, and more,
  - Owners can manage pull request reviews in the organization.
- You would need write access to the main branch to open a pull request.

45d. Deployment pipeline - Code Owners

- If you want Pull Requests to automatically have somebody review code, you can use the CODEOWNERS file.
  - You create a CODEOWNERS file in:
    - the .github folder,
    - the root folder, or
    - the docs folder.
    - If you have multiple CODEOWNERS files, it will use the first one (in the above order).
  - The file must be under 3 Mb.
- Create the CODEOWNERS file in the following format:
  - Filename space/tab OwnerName. For example:
    - *.txt @myusername
  - You can also add comments starting with a #, either at the start of the line or the middle.

- The Filename can use wildcards, such as:
    - * - all files in the root folder and all subfolders
    - *.js – all files with a js extension, in the root folder and all subfolders
    - docs/* – all files in the docs folder (but not subfolders)
    - /docs/ – all files in the docs folder and also subfolders
- In the event of multiple Filenames which match, the last match takes priority.
- The OwnerName can be:
    - A username – @username
    - A team name - @organizationname/team-name
    - An email address – myemail@address.com (it cannot be used in managed user accounts).
    - There are be multiple OwnerNames, separated by a space. Approval from only one OwnerName is required, not all of them.
- You should define an OwnerName for the CODEOWNERS file itself, to ensure that it cannot be changed without authorization.

Integrate SQL solutions with Azure services

Methodology - DAB

- Install Visual Studio Code.
- Check if npm is installed. Go to View -Terminal and enter
    - npm -v
- If it is not installed, go to https://nodejs.org and install it.
    - (No need to install “chocolatey”).
    - Then restart VS code and check using “npm –v” to see if it is installed.
- If you are getting the error “File...npm.ps1 cannot be loaded because running scripts is disabled on this system”, then run:
    - Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
- Is dotnet installed?

    - dotnet --info
- If so, install the .NET SDK (LTS) from
    - https://aka.ms/dotnet/download
    - At the time of recording, .NET 10.0 SDK has the “Long Term Support” (and is not in preview).
- Then install the Data API Builder CLI
    - dotnet tool install --global Microsoft.DataApiBuilder
- Then install Azure SQL Database
    - Enter a Subscription and Resource Group
    - Configure the server – use SQL Authentication,
    - Use the “Basic” service tier,
    - In the Networking tab,
        - change the Connectivity method to “Public endpoint”,
        - Enable “Allow Azure services and resources to access this server”, and
        - Enable “Add current client IP address”.
    - In Additional tab:
        - Change “Use existing data” to “Sample”.
- In Visual Studio Code:
    - Create a new folder.
    - Run the following commands. This requires Microsoft .NET Runtime version 8. If you do not have it installed, install it, then close Visual Studio Code and re-open it.
        - dab init --database-type mssql --config dab-config.json --connection-string "Server=...;Connection Timeout=30;"
            - The connection-string should be taken from the Azure SQL Database (Settings – Connection strings).
        - dab add Address --source SalesLT.Address --permissions "anonymous:read“

46. Create configuration files for Data API builder (DAB)
    - dab add Customer --source SalesLT.Customer --permissions "anonymous:read"
    - dab add CustomerAddress --source SalesLT.CustomerAddress --permissions "anonymous:read"
        - If the table, query or stored procedure has already been added, you can use “dab update” to update it.
    - In the dab-config.json file, change mode from production to development.
    - dab start
        - When you want to end it, press Ctrl and C.

- In a web browser, enter:
    - http://localhost:5000/health
        - This will only work if mode is development.
    - http://localhost:5000/api/Address (this uses the REST API).
        - This will give you the first 100 records.
        - The last bit is a “nextLink”, which gives you a link to the next 100 records (paging).
- You can also use http://localhost:5000/swagger to:
    - GET – returns all entities or an entity,
    - PUT – replace/create an entity,
    - PATCH – update/create an entity,
    - DELETE – delete an entity,
    - POST – create an entity.

46. Create configuration files for Data API builder (DAB)

- You can create multiple configuration environments – for example, one for Development and one for Production.
- You create a file called “.env” with the connection strings:
    - devconnectionstring=Server=tcp:127.0.0.1,1433;User ID=<username>;Password=<password>;
- Note that there is no space before or after the = sign.

- And then use the appropriate environment string in the dab-config.json file:

```
"data-source": {
  "connection-string": "@env(' devconnectionstring ')"
}
```

- Note that:
    - “@env” must be in lower case.
- When you deploy the Container App, you will need to add an environment variable. In the Container app:
    - Go to Security – Secrets and add a key/value pair secret. This is used to protect sensitive data like passwords and connection strings.
    - Go to Application – Containers and the Environment variables tab. Add a new environment variable:
        - The name should be the same name as the environment variable in Visual Studio Code.
        - Source should say “Reference a secret”.
        - Value should be the secret name.

47. Configure entities for REST and GraphQL, including data caching, pagination, searching, and filtering

- Click “Try it out” to enter values.
- You can use:
    - $select to select the fields (columns) to be returned.
        - For example, AddressLine1,City
    - $filter it using OData predicates:
        - eq = equals,
        - ne = not equals to,
        - and = both conditions need to be true,
        - or = either condition can be true,
        - gt = greater than,
        - lt = less than.

        - For example, AddressID gt 25
        - http://localhost:5000/api/Address?$filter=AddressID eq 25
        - http://localhost:5000/api/Address?$filter=AddressID gt 25
    - $orderby to sort
        - For example, AddressID desc
- You can also get a page of data by using first (page size) and after (the previous page)
    - http://localhost:5000/api/Address?$first=10
    - If it ends with "nextLink":http://localhost:5000/api/Address?$first=10&$after=ABCD then you can use:
        - http://localhost:5000/api/Address?$first=10&$after=ABCD to connect to the next page.
- http://localhost:5000/graphql when mode is “development” (this uses GraphiQL UI). In “production”, this testbed is not used, but you can use Postman Desktop Agent. You can run queries like:

```
query AddressList {
  addresses
  { items
    { AddressID, City}}
}
```

- in items, you can choose which columns to return
    - for example { column1 column2 } (no commas)
    - there is no * wildcard equivalent – you do need to state each field.
    - You can alias fields, such as bookTitle: title takes the “title” column and renames it as “bookTitle”.
    - You can also have additional tables in items – this does an INNER JOIN – for example { column1 column2 different_table {column3 column4 } }
- in filter, you can reduce the number of rows using OData predicates:
    - eq/neq – equal/not equal. For example:

        - column1 {eq: 5},
        - column2 {neq: "Foundation"}
    - gt/gte – greater [or equal to]
    - lt/lte – less than [or equal to]
    - and/or. For example:
        - or: [{column1 {eq: 5} }, { column2 {neq: "Foundation"} }]
    - contains/notContains – anywhere within a string
    - startsWith/endsWith – at the beginning/end of a string
    - in. For example:
        - (filter: { category: { in: [“Foundation”, “Advanced”] } })
    - isNull. For example:
        - (filter: { category: { isNull: false } })
- groupBy combines by a particular field(s). For example:
    - groupBy: { fields: ["column1“] }
- orderBy orders the query before using first or after.
    - By default, DAB will sort ascending by the primary key.
    - You can order by DESC or ASC.
    - For example: books(orderBy: { year: DESC, title: ASC }, first: 5) { items { id title year } }
- first allows you to return a specified number of rows (in a page).
    - The default page is 100,
    - The maximum page size defaults to 100,000.
    - For example: books(first: 5)
- after allows you to take the end of a previous page, and continue it.
    - You can add in a query:
        - hasNextPage – to find out whether additional data exists, and
        - endCursor – to get the next token.
    - You might get the response

- { "data" : { ... "hasNextPage" : true, "endCursor": "ABCDE“ } }

- then you can use, in the next query.

- query { books(first: 3, after: "ABCDE") { items { ... } } }

- aggregates allows you to do a sum, average, min, max and/or count.

- For example:
  { book { items { ... }
  aggregates { column1 { sum average min max } } } }

- You can also use “distinct: true” for count, and “having: { }” to filter after aggregation.

- You can use:

- totalcost: sum (totalcost becomes an alias),

- sum(having: {gt: 10000 }) , and

- count(distinct: true)

- Additional query/stored procedures to add:

dab add CustomerAndAddress --source SalesLT.vCustomerAndAddress --source.key-fields CustomerID --permissions "anonymous:read"

- Create Stored Procedure:

CREATE PROC SalesLT.Product700 AS

select *

from SalesLT.Product

WHERE ProductID BETWEEN 700 AND 799;

- Reference that Stored Procedure:

dab add Product700 --source SalesLT.Product700 --source.type "stored-procedure" --permissions "anonymous:*"

- Create Stored Procedure:

CREATE PROC SalesLT.ProductProc (@FirstProd int = 700, @LastProd int = 799) AS

select *

from SalesLT.Product

WHERE ProductID BETWEEN @FirstProd AND @LastProd;

- Reference that Stored Procedure:

dab add ProductProc --source SalesLT.ProductProc --permissions "anonymous:*" --source.type "stored-procedure" --parameters.name "FirstProd,LastProd" --parameters.description “First Product,Last Product" --parameters.required "true,true" --parameters.default “700,700"

    - Permissions are required to be added in DAB 1.x, and are more optional in DAB 2 (it defaults to “Anonymous”.
        - They could be Anonymous, Authenticated, or custom roles from your identity provider.

```
"permissions": [
 {
  "role": <"Anonymous" | "Authenticated" | "custom-role">,
  "actions": [ <string> ]
 }
]
```

        - For example:
            - For tables or views: { "role": "anonymous", "actions": ["read"] }
            - For stored procedures, either "actions" : ["*"] or "actions": "*" (without the brackets).
        - actions can be:
            - For tables and views: create, read, update or delete one row in REST API, and one or more rows in GraphQL.
            - For “stored-procedure”, execute (runs the stored procedure).
            - * - all actions as above.
    - You can also create more complicated actions, which includes what fields to include/exclude, and what rows to include:

```
"actions": [
    {
      "action": "read",
```



```
    "fields": {
      "include": ["*"],
      "exclude": ["last_login"]
    },
    "policy": {
      "database": "@item.IsAdmin eq false"
    }
  }
]
```

    - The policy uses OData predicates:
        - eq = equals,
        - ne = not equals to,
        - and = both conditions need to be true,
        - or = either condition can be true,
        - gt = greater than,
        - lt = less than.

- Configure entities for REST and GraphQL

```
        "actions": [
  {
    "action": "read",
    "fields": {
      "include": ["*"],
      "exclude": ["last_login"]
    },
    "policy": {
      "database": "@item.IsAdmin eq false"
    }
  }
```


]

- Configure entities for REST and GraphQL - Entities
- Configure entities for REST and GraphQL - Permissions
- Configure entities for REST and GraphQL - Fields
    - The optional “cache” allows for previous results to be retained in memory, so if the results can be reused, they will be. It includes:
        - enabled – true or false (the default is false),
        - ttl-seconds – time to live (how long the data is to be retained); the default is 5 seconds.
        - Level:
            - L1 is in-memory cache only. It is the quickest, but is not shared across instances.
            - L1L2 is in-memory cache and distributed (Redis) cache. It is shared across the scaled-out instances, and is the default.
- graphql.type shows the singular and plural names for an entity.
    - They are optional when object “type” is specified as a string.
    - If object “type” is specified as an object, then “singular” is required.

```
"type": {
    "singular": "User",
    "plural": "Users"
  }
```

- graphql.operation is optional and is used for “stored-procedure”, and can be query (read-only) or mutation (read/write).
    - 47. Configure entities - data caching

```
{
 "entities": {
  "Author": {
   "cache": {
    "enabled": true,
```



```
    "ttl-seconds": 30,
    "level": "L1"
   }
  }
 }
}
```

48. Configure REST or GraphQL endpoints

- graphql.allow-introspection:true allows you to learn about an API's schema
    - dab configure --runtime.graphql.allow-introspection true
    - You can query it using:

      query {__schema { types {name kind} } }
- rest.path is used if you want a custom route for the REST endpoint.
- rest.methods says what methods the stored-procedure should allow:
    - The defaults are GET and POST.
    - Others are PUT, PATCH and DELETE.
- Configure REST or GraphQL endpoints

49. Expose database objects, stored procedures, and views, including GraphQL relationships

- The entities (tables) are contained within:

  "entities": {

  }
- You then name the entity, and then give the source, which includes:
    - the “object”
        - You don't need to specify the schema if it belongs to the [dbo] schema.
        - You can add square brackets around object names.
    - “source.type” (table, view or “stored-procedure”).
    - If the “source.type” is “view”, then you need to use the “fields” array.

- In DAB 1.x you could use “key-fields”, but that is being deprecated in DAB 2.

```
"Address": {
 "source": {
  "object": "SalesLT.Address",
  "type": "table"
 },
```

- Parameters when type is “stored-procedure”:
  - name (don't use the @ prefix), and optionally
  - required: whether the parameter is needed – true or false,
  - default: the value to be used if the parameter is not supplied
  - description: of the parameter.

```
"parameters": [
        {
                "name": "id",
                "required": true,
                "default": null,
                "description": "My description"
        }
]
```

- You can add relationships between the entities after the entities have been added (using dab add).
- You would dab update to add the relationships. For example:

  dab update Customer --relationship CustomerAddresses --target.entity CustomerAddress --cardinality many --relationship.fields "CustomerID:CustomerID"

  dab update CustomerAddress --relationship Address --target.entity Address --cardinality one --relationship.fields "AddressID:AddressID"
- It is used by GraphQL.

- It requires a unique “<relationship-name>”, which needs to be unique for all that entity's relationships.
- Within it, it also requires:
  - “cardinality” – either:
    - “one”, for one-to-one relationships, or the “many” side of a many-to-one relationship, or
    - “many”, for the “one” side of a one-to-many relationship, or for a many-to-many relationship.
  - target.entity – the name of the target entity.
- You can also use:
  - source.fields and target.fields – these needs to be enclosed in hard brackets and speech marks.
- For many-to-many relationships, you would need a third table, called a linking object, between the two entities, and have a:
  - linking.object string, and
  - linking.source.fields and linking.target.fields.

- 49. Expose database objects, stored procedures, and views, including GraphQL relationships
- You can then query multiple tables using GraphQL, and filter using Odata predicates:

```
query CustomersWithAddresses {
 customers
(
  filter: {
   CustomerID: { eq: 29485 }
  }
 )
{
  items {
   CustomerID
```



```
    FirstName

    LastName


    CustomerAddresses {
     items {
      AddressType


      Address {
       AddressID
       AddressLine1
       AddressLine2
       City
       StateProvince
       PostalCode
       CountryRegion
      }
     }
    }
   }
  }
}
```

50. Configure and implement DAB deployment

- In Visual Studio Code, create a new file called Dockerfile (one word, no extension) with the following:

  ```
  FROM mcr.microsoft.com/azure-databases/data-api-builder:latest

  WORKDIR /App

  COPY dab-config.json /App/dab-config.json

  EXPOSE 5000
  ```



```
CMD ["dab", "start", "--config", "dab-config.json", "--host", "0.0.0.0"]
```

- This is used to create the container image:
    - The first line shows the base image:
        - mcr.Microsoft.com is the Microsoft Container Registry
    - The second line changes the Working Directory for the container to /App.
    - The third line copies the local configuration file to the /App folder in the container.
    - The fourth line declares the intent that the container will listen on TCP port 5000.
    - The fifth line runs the dab start command.
        - host 0.0.0.0 means that the container will listen on all network interfaces.
- In Azure, create a container registry.
    - Go to https://portal.azure.com
    - Enter “container registry” in the search menu at the top, and go to the SERVICE “Container registries”.
    - Click Create.
    - Select a Subscription and Resource Group, and enter a registry name (which can only use alphanumeric characters).
    - Choose the “Basic” Pricing plan.
        - This is 16.7 cents per day.
        - Standard is 66.7 cents per day.
        - Premium is $1.67 cents per day.
    - Go to “Review + create” and create the container registry.
- In Visual Studio Code, interact with Azure:
    - Download the Azure CLI
        - winget install --exact --id Microsoft.AzureCLI
        - And then close and re-open Visual Studio Code.
    - Or check that it is installed

    - az version
- Then log into Azure
    - az login
    - and select the appropriate subscription.
- Then build the container (the dot at the end assumes that you are in the appropriate directory).
    - az acr build --registry <<Name_Of_Container>> --image dab:latest .
    - The Name of the container can end (but does not need to end) with .azurecr.io.
    - The period at the end is the source location; this uses the current folder.

- You can then deploy it onto:
    - an Azure Container Instance,
    - Azure Kubernetes Services cluster, or
    - A Container App.
- In Azure, you can create a Container App.
    - Go to https://portal.azure.com
    - Enter “container app” in the search menu at the top, and go to the SERVICE “Container apps” (not a similarly-named service).
    - Click Create – “+ Container App”.
    - In the Basics section:
        - Select a Subscription and Resource Group, and enter a Container App name (which must contain numbers, lowercase letters or a hyphen).
        - In Deployment source, select “Container image”.
        - Select an appropriate region.
        - If necessary, create a new Container Apps environment.
    - In the Container tab:

        - In the Container details, select “Azure Container Registry”, and select the Subscription, Registry, image and Image tag.
    - In the Ingress tab:
        - Check “Ingress”.
        - Change “Ingress traffic” to “Accepting traffic from anywhere”.
        - In “Target port”, select 5000 (the default for DAB).
    - Go to Review + Create, and then Create.
    - You can then test the app by copying the Application URL and pasting it into a new Url or using an application such as Postman to test it.
        - If you have the word “internal” in the Application UR, then you did not change “Ingress traffic” to “Accepting traffic from anywhere” (go to Networking – Ingress to add it).
    - Pay as you go price is around US$2 per day.

51a. Recommend Azure Monitor configurations: Application Insights

- In Data API Builder (DAB), you can add Azure Application Insights.
    - It is an Application Performance Monitoring (APM) service which captures requests, traces, exceptions and performance metrics.
    - It allows you to:
        - monitor behavior,
        - diagnose problems, and
        - optimize DAB's performance.
- In the Azure Portal, you can create an Application Insights instance.
- Afterwards, copy the Connection string.
- In Visual Studio code, enter:
    - dab add-telemetry --app-insights-enabled true --app-insights-conn-string "<<Connection_String>>"
- When you go to dab start and run a query, you can then go back to the Application Insights service, and go to:
    - Monitoring - Logs and look at the traces or requests tables, or
    - Investigate - Live metrics.

51b. Recommend Azure Monitor configurations: Log Analytics

- Log Analytics can store logs from Data API Builder.
    - It can help you meet requirements on compliance, governance and observability.
    - Unlike Application Insights, it concentrates more on logs aggregation.
- In the Azure Portal, search for and create:
    - a data collection endpoint, and
    - a new Log Analytics workspace in the same region.
- In the Log Analytics workspace, go to Settings - Tables, and create a new Table:
    - Give the table name a name ending with _CL (this is needed for the Logs Ingestion API).
    - Select a Table Plan:
        - Auxiliary is the cheapest, and is good for auditing and compliance.
        - Basic is for medium-touch data,
        - Analytics is for high-value data.
    - Create a new Data collection rule.
    - Use the Data collection endpoint.
    - Then create a table to store the data with Time, LogLevel, Message, Component and Identifier strings.
- In Visual Studio Code, you can then configure:
    - dab configure --runtime.telemetry.azure-log-analytics
        - .enabled true,
        - .auth.dcr-immutable-id for the data collection rule
        - .auth.dce-endpoint for the data collection endpoint, and
        - .auth.custom-table-name for the table name.

(51c). Recommend Azure Monitor configurations: OpenTelemetry

- You can also use the open source OpenTelemetry for tracing and metrics.
- DAB creates OpenTelemetry "activities" for:
    - Incoming HTTP requests from REST endpoints,

    - GraphQL operations,
    - Database queries,
    - Internal middleware steps (such as request handling and error tracking), and
    - MCP tool execution.
- It tracks items including the:
    - api.type,
    - data-source.type and .name,
    - user.role and user-agent
    - action-type (for example, create, or GraphQL), and
    - http.method, .url, .querystring and status.code.
- You can use dab add-telemetry to add a --otel (Open Telemetry):
    - --otel-enabled - true/false,
    - --otel-headers,
    - --otel-protocol,
    - --otel-service-name, and
    - --otel-endpoint.

52a. Handle changes by using change event streaming (CES)

- Change Event Streaming (CES) was introduced in SQL Server 2025.
    - But not on SQL Server 2025 on Linux.
- It streams SQL Server data changes directly into Azure Event Hubs.
    - Azure Event Hubs is a high throughput data streaming service.
    - Data changes are updates, inserts and deletes.
    - Data which has been captured include schema, previous and new values.
        - It does not include DDL operations (schema changes), but they can still be done.
            - Note: adding/removing primary key constraints are not allowed.

    - It also does not include historic changes made before CES was enabled.
    - It will skip columns of the types geography, geometry, image, json, rowversion / timestamp, sql_variant, text / ntext, vector, xml or User-defined types (UDT)
  - Data is sent as a CloudEvent using JSON or Avro Binary.
  - When enabled, renaming tables or columns is not allowed, and neither is TRUNCATE TABLE (and will fail).
  - A table can only be included in one stream group (so cannot go to multiple destinations).
  - Tables must be user tables (not system tables), and cannot use:
    - Clustered columnstore indexes
    - Temporal history tables or ledger history tables
    - Always Encrypted
    - In-memory OLTP (memory-optimized tables)
    - Graph tables
    - External tables
  - The database needs to be using the Full Recovery model.
- Change Event Streaming (CES):
  - requires minimal overhead,
  - integrates easily,
  - can be used to synchronize data across systems,
  - can be used for real-time analytics,
  - can be used to audit and monitor.
- Its advantages are:
  - Scalability - for high-throughput,
  - Decoupling - it is done separately from the database,
  - Multi-consumer support - it allows multiple consumers to use the same data stream,
  - Real-time integration - between OLTP and other systems.

- To create an Azure Event Hubs:
    - Go to the Microsoft Azure Portal (portal.azure.com),
    - Click on "+ Create",
    - Enter the Subscription, Resource group, Namespace name, Region, Pricing tier and Throughput Units.
        - Click on "Browse the available plans and their features" for details.
        - You can also change information in the Advanced and Networking tabs, if required.
        - Go to the "Review + create" tab, and click Create.
- You would then need to create an access control, either:
    - A Shared access policy, or
    - A Microsoft Entra managed identity.
- In SQL Server, you need to:
    - Ensure the database uses the Full recovery model,
    - Create a master key and a database scoped credential,
    - Enable event streaming,
    - Create the event stream group, and
    - Add at least table to the event stream group.
- Once event streaming is enabled then the following will show 1 for that database:
    - SELECT name, is_event_stream_enabled FROM sys.databases;

52b. Handle changes by using change data capture (CDC)

- Change data capture (CDC) records when tables/rows have been modified. You cannot use it on an Azure SQL Database where:
    - Free, Basic or Standard tier Single Database (S0,S1,S2), and
    - Database in Elastic pool with max eDTUs less than 100 or max vCore less than 1
- To enable CDC on an Azure SQL Database, you use the following command:
    - EXEC sys.sp_cdc_enable_db;

    - GO
- Once enabled, in the catalog view sys.databases, the column is_cdc_enabled is 1.
- When CDC is enabled, it then creates 6 CDC system tables:
    - cdc.captured_columns - used when you specific which columns are to be captured (as opposed to "all of them"),
    - cdc.change_tables - one row for each table which is captured
    - cdc.ddl_history - capturing Data Definition Language changes (when the table structure is changed),
    - cdc.index_columns - capturing index columns for a change table.
    - cdc.lsn_time_mapping - when changes were made. lsn is Log Sequence Number.
    - dbo.systranschemas.
- However, this does not capture any data. In addition, you need to enable CDC for a table:
    - EXEC sys.sp_cdc_enable_table
    - @source_schema = N'SchemaName',
    - @source_name = N'TableName',
    - @role_name = NULL;
- This then creates a System table "cdc.<NameOfTable>_CT". This captures changes:
    - 1 row for an insert operation,
    - 1 row for a delete operation, and
    - 2 rows for an update operation.
- Microsoft recommends not querying these tables directly - you could use the CDC system function cdc.fn_cdc_get_all_changes_dbo_tblReviews instead.
- You can then use review the changes made to assertation if any other changes are to be made. For example, if you have embeddings, they may have to be NULLed and then regenerated.

52c. Handle changes by using Change Tracking

- Change Tracking shows whether there has been a change to a row.

- To enable it on a database, use:
    - ALTER DATABASE NameOfDatabase
    - SET CHANGE_TRACKING = ON
- You can also add:
    - (CHANGE_RETENTION = 3 DAYS, AUTO_CLEANUP = ON)
    - CHANGE_RETENTION specifies the minimum period for which change tracking information is kept.
    - AUTO_CLEANUP enables/disables the periodic removal of old change tracking data.
- In addition, you need to enable it on a table:
    - ALTER TABLE NameOfTable
    - ENABLE CHANGE_TRACKING
        - You can also use DISABLE CHANGE_TRACKING
    - WITH (TRACK_COLUMNS_UPDATED = ON)
    - This last argument allows Change Tracking to record not just that a change has happened to a row, but also to a column.
- To check what databases and tables have change tracking enabled, you can use these catalog views:
    - sys.change_tracking_databases
    - sys.change_tracking_tables
        - SYS_CHANGE_COLUMNS is not NULL if TRACK_COLUMNS_UPDATED = ON and the row has been updated.
        - It shows a different value based on which column(s) have been updated.

52c. CDC vs Change Tracking

- Using either CDC or Change Tracking has the following advanced:
    - Built into SQL Server - reduced development time,
    - No schema changes needed,
    - Built in clean-up mechanism,
    - Built in SQL functions,

    - Low overhead for DML operations (reading/writing).

- Change data capture provides historical change data:
    - The fact that DML changes were made, AND
    - The actual data that was changed.
- Change tracking only provides the fact that changes were made.
    - It is more of a lightweight solution.

52d. Handle changes by using Azure Functions with SQL trigger binding

- If you want an action which happen when changes are made to a database table, you can use Azure Functions with SQL trigger binding.
    - Changes are when a row is created, updated or deleted in a table which is being monitored.
- First of all, you need Change Tracking (CT) to be enabled for at least one table (see topic 52c).
    - You can also enable CDC in your database, but it is irrelevant - it is not needed for Azure Functions with SQL trigger binding.
- The Azure SQL trigger binding then checks for changes, and runs the Azure Function when there are changes.
    - If data is encrypted, then the encrypted data is not exposed to the function. However, there is a notification that data has been changed instead.
- You would then write a function in C#, Java, JavaScript, PowerShell or Python to do actions based on that changed data.
    - Because you are writing code, this means that you can create a wide range of automated responses to table changes.
    - You can use Visual Studio Code if it has the MSSQL extension by going to the Command Palette (Ctrl/Command+Shift+P) and entering "Create Azure Function with SQL binding" (this creates it in C#).

52e. Handle changes by using Azure Logic Apps

- Azure Logic Apps is able to detect changes (with a little bit of help) in a Database, and then run actions.
    - You could also create a Power Automate flow.

- In the SQL Server Database table, add a rowversion column:
    - ALTER TABLE NameOfTable
    - ADD RowVer rowversion
- This helps Azure Logic Apps track any changes.
    - You may also be to add a ModifiedDate column or other columns that shows a clear change.
- To create the Logic App instance:
    - go to the Azure portal (portal.azure.com),
    - search for Logic App.
    - Click on "+ Create".
    - Select a hosting option:
        - Consumption.
            - This is a "pay what you use" model, and is the easiest to use for beginners.
            - A logic app can have only one workflow.
            - Standard
                - This allows for more built-in connectors, more control, more fine-tuning capability.
                - It is available as:
                    - a single-tenant Workflow Service Plan,
                    - an isolated App Service Environment, or
                    - on your own on-premises infrastructure (Hybrid).
    - Enter the subscription, resource group, logic app name and region.
        - Depending on the hosting option, there may be additional options.
- Once created, you can click Edit and:
    - Click on "Add a trigger":
        - Search for "SQL Server".
        - Click on "When an item is modified".

        - Optionally change the Connection Name.
        - Change the Authentication Type (say, to SQL Server Authentication).
        - Enter the SQL Server Name (for example, <DatabaseName>.database.windows.net)
        - Enter the SQL Database Name, and credentials.
        - You can also click on "How often do you want to check for items?" and change the interval.
            - Do not make it too quick. For example, 2 seconds, which would be fine for testing
    - Click on the + sign and select "Add an action".
        - This could be in SQL Server or elsewhere.
            - For example, "Execute stored procedure" - dbo.uspLogError.
- Once you have created your Logic App, click on "Run".
- You can then make changes to the Database table, and see that the Azure Logic App will run.
- If you no longer want the Logic App to run, click on "Disable".

Design and implement models and embeddings

53. Evaluate external models, including multimodal, multilanguage, sizes, and structured output

- You can view OpenAI's external models at https://developers.openai.com/, and click on API then Models.
    - You can then click on "View all" or "Compare models".
- When you find a model, you will see at the top of the window capabilities including:
    - Reasoning,
    - Speed,
    - Price (Input and Output) per 1 million tokens

    - Further down, there is a price for "Cached input".
  - Types of Input/Output - Text, image, audio and video (if a model has more than one type of input, then that is called "multimodal").
- It also shows the size of:
  - the context window (the number of input tokens which is part of the conversation)
    - If you exceed this, then future input will "forget" earlier parts of the conversation, as they will not be sent.
  - the maximum number of output tokens.
  - A token is a small part of the input, roughly around a syllable, and is the basis of charging.
- You can also look at the endpoints, which including:
  - Chat Completions,
  - Responses,
  - Embeddings
    - for example, text-embedding-large, -small and ada.
    - Cohere also have embed-English and embed-multilingual model.
  - Image generation/edit, and
  - Videos.
- The features include:
  - Structured outputs - this creates responses in a JSON format.
- You can check what languages are used in the Multilingual AI Model Benchmark.

54. Create and manage external models

- You can create external models in various places, including in Microsoft Foundry.
- The models include models from:
  - Microsoft,
  - OpenAI,
  - DeepSeek,
  - Hugging Face, and

    - Meta.

- To create a model, go to Microsoft Foundry (https://ai.azure.com).
    - You can use either the Classic or New Foundry (there is a switch at the top middle of the window).
- To deploy a model in Classic Foundry:
    - Go to Model catalog (on the left-hand side).
    - Search for and click on a model.
    - Click on "Use this model".
- To deploy a model in New Foundry:
    - First you may need to create a project.
    - Then go to Discover - Models and search for a model (for example, "GPT" or "embedding").
    - Click on a model, and then Deploy - Default settings.
    - Your model will then be available in Build - Models - Deployments.

55a. Choose an embedding maintenance method, including table triggers

- There are two types of table triggers, which can be called when an INSERT, UPDATE, MERGE or DELETE is done:
    - INSTEAD OF - this replaces the action with a custom action, on a table or view.
    - FOR/AFTER trigger - this adds an additional action after the original action has happened on a table (not a view).
- The syntax of an AFTER trigger is:
    - CREATE/ALTER TRIGGER [Schema].TriggerName ON TableName
    - AFTER INSERT, UPDATE, DELETE [as appropriate]
    - AS statement.
- You can use this to SET a vector column to NULL where the main column has been UPDATEd.
    - You can run a SELECT statement, EXECUTE a stored procedure, or more using BEGIN ... END.

55g. Choose an embedding maintenance method, including Microsoft Foundry

- Microsoft Foundry is the host of the AI model which generates the embeddings.
- It does not store the data itself.

56. Identify which columns to include in embeddings

- An embedding is a vector of decimal numbers.
    - These can be used to see if two embeddings are semantically similar.
- The maximum length of input text for OpenAI embedding models is 8,192 tokens
    - for text input, for a token think "syllables" or around 4 characters (though that is not necessarily the case).
    - It could also represent images, audio, or structured data.
- If you are sending multiple inputs, then you can send a maximum number of 2,048 at any one time.
- There will also be limits in deployments regarding the number of tokens per minute.
- Columns which are to be included in embeddings should be:
    - unstructured text data those that have meaning and whose meaning you want to query,
    - you may or may not want to include:
        - metadata (title, author, category, tags, language) - this may (or may not) be more often used in a text search rather than a semantic search. Consider how you would use it.
        - unique identifiers, which may be more than just a simple number, but which might include other fields,
    - rather than be numerical/textual data, or strings which do not have meaning.

57. Design and implement chunks for embeddings

- Data is divided into chunks, before it is sent for embedding.
- This means that the data can be divided into a size which is acceptable for a model, based on its maximum input size.
    - For smaller pieces of text, it would probably be contained in one chunk.

- Microsoft recommends:
    - a chunk size of 512 tokens (about 2,000 characters), and
    - an overlap of 25% between chunks (about 128 tokens).
        - The overlap allows for meaning not to be stopped between chunks.
- However:
    - You may want to have larger chunks to better keep the structure of sentences or user queries.
    - You may wish to have a look at the model performance guidelines.
- You can divide longer documents into chunks by using:
    - SELECT * FROM AI_GENERATE_CHUNKS(SOURCE = TextExpression,CHUNK_TYPE=FIXED)
    - SELECT * FROM SourceTable CROSS APPLY AI_GENERATE_CHUNKS(SOURCE = TextExpression,CHUNK_TYPE=FIXED)
    - CHUNK_TYPE=FIXED is the only option.
    - Optional arguments are:
        - , CHUNK_SIZE = NumberOfCharacters,
        - , OVERLAP = Percent between 0 and 50 (the default is 0)
        - , ENABLE_CHUNK_SET_ID = 1 or 0. 1 would return a column which has a number which can be used to group chunks to the same source.
- It returns:
    - chunk - the chunked text,
    - chunk_order - the order that the chunks were processed, starting with 1,
    - chunk_offset - the position of the chunk in the source data,
    - chunk_length - the number of characters, and
    - optionally chunk_set_id - each document/row has the same chunk_set_id.

58. Generate embeddings

- To create embeddings, you will need:
    - The Azure OpenAI/Foundry endpoint.

    - Go to the Foundry/Azure OpenAI resource (not the model),
    - Go to Resource Management - Keys and Endpoint, and copy the endpoint URL in a Notepad. (It should end ".openai.azure.com".)
- A Managed Identity for your Azure SQL Server:
    - Go to the Azure SQL Server (not Database),
    - Go to Security - Identity,
    - Change the Status for "System assigned managed identity" to On.
    - Click Save.
- Allow that Managed Identity access to the Foundry resource:
    - Go to your Foundry resource (not the model),
    - Click on "Access control (IAM)",
    - Click on "+ Add" - "Add role assignment",
    - In the Role tab, select "Cognitive Services OpenAI User", and click Next,
    - In the Members tab, select "Managed Identity", and click "+Select members",
    - In the Select managed identities pane, change "Managed Identity" to "SQL server", select the SQL Server, and click Select.
    - Click "Review + assign" twice.
- In SSMS, Add a Master Key Encryption. This protects the Database Scoped Credential:

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD =
'StrongP@assword123';
```

- Then create a Database Scoped Credential. The name of it should be the name of the Open AI Endpoint you have copied earlier in hard brackets [ ].

```
CREATE DATABASE SCOPED CREDENTIAL [https://___ai.azure.com]
 WITH IDENTITY = 'Managed Identity', SECRET =
'{"resourceid":"https://cognitiveservices.azure.com"}';
```

- Then create an external model reference. You should use the same Database Scoped Credential (in hard brackets) you have previously created:


```
CREATE EXTERNAL MODEL EmbeddingModel

WITH (

   LOCATION = 'https://___ai.azure.com /openai/deployments/text-embedding-3-small/embeddings?api-version=2024-10-21',

   API_FORMAT = 'Azure OpenAI',

   MODEL_TYPE = EMBEDDINGS,

   MODEL = 'text-embedding-3-small',

   CREDENTIAL = [https://___ai.azure.com]

);

GO
```

- You can then test it using AI_GENERATE_EMBEDDINGS:
    - SELECT AI_GENERATE_EMBEDDINGS('This product was great.' USE MODEL EmbeddingModel)

Design and implement intelligent search

59. Choose from full-text, semantic vector, and hybrid search

- Full-text search, which has been around for over 13 years, allows you to search through a text column for:
    - Specific words or phrases (simple term),
    - Words/phrases starting with the search term (prefix term)
        - for example, a prefix term “beg*” can find “begin” and “begged”.
    - Inflection forms of a term, to find different verb tenses or conjugations, or singular/plural versions of a noun (generation term),
    - A word/phrase being near another word/phrase (proximity term),
    - Words with a similar meaning (thesaurus or synonym),
    - An analysis of how close a word/phrase is to other phrases (weighted term).
- Semantic vector search aims to “understand” the text.
    - It is a newer version of search using AI and machine learning.

    - It evaluates chunks (which could be words, phrases, or parts of words) to see how similar they are to other words using hundreds of different comparisons.
    - You can use it when you need SQL Server to search for meaning, not exact words or similar.
    - For example, searching for kitten may retrieve cat and puppy, with different levels of matching.
- Hybrid search does both full-text and vector search, and give you combined results.
    - This is good when the developer is unsure whether to do an exact text match or a semantic match.
    - Because it does both searches, it does take longer.

60. Implement full-text search

- If you are implementing full-text search in on-premises SQL Server, or on a Virtual Machine, first of all the Full-Text Search component will need to be installed when installing SQL Server, or when adding new components.
- Next, you need to create a full-text catalog.
    - This is a container for full-text indexes.
    - Run the following T-SQL command:
        - CREATE FULLTEXT CATALOG NameOfCatalogue;
    - You can also add before the semi-colon:
        - WITH ACCENT_SENSITIVITY = ON/OFF
        - AS DEFAULT
            - This makes this catalog the default catalog,
            - Having a default catalog means that any full-text indexes you create will automatically use that catalog, and you do not need to specify a catalog.
        - AUTHORIZATION owner_name
            - Either the current user or one that you have impersonate permissions, or a different user if you are the database owner or the system administrator.

    - The owner_name must have TAKE OWNERSHIP permission on this full-text catalog.

- You can query what full-text catalogs you have by using
    - SELECT * from sys.fulltext_catalogs;
    - The first catalog will have ID 5 (and the next 6, then 7 and so on).
- Subsequently, you can DROP or ALTER FULL TEXT CATALOG.
    - With ALTER, you can use REBUILD, REBUILD WITH ACCENT_SENSITIVITY = ON/OFF, REORGANIZE, and AS DEFAULT
        - REBUILD drops the current catalog and creates a new one.This can only happen if no other user is currently using the full-text catalog.
        - REORGANISE optimizes the existing one. This can be useful if changes are often made to the indexed data.

- Next, you can then create a full-text index
    - CREATE FULLTEXT INDEX ON NameOfTable
    - (Column1, column2…)
    - KEY INDEX IndexName
- Alternatively, in SSMS:
    - Right-hand click on the relevant database in Object Explorer.
    - Go to Full-Text index, and then Properties.
- The columns must be of either type char, varchar, nchar, nvarchar, text, ntext, xml, image or varbinary(max).
    - These last 2 are used to hold files such as Office documents or PDF – that is beyond the scope of this course.
- After each column, you can say LANGUAGE NameOfLanguage.
    - NameOfLanguage can be a string, integer, or hexadecimal value relating to the language's locale identification (LCID). For example, 1033 is American English.
    - You will need to have certain resources enabled for this language – for example, word breakers and stemmers.

- KEY INDEX needs to be the unique key index for the table. It needs to be a single-key (not compound) index name on a NOT NULLable column.
    - Microsoft recommends an integer data type for this index.
- Afterwards, you can specify its catalog.
    - ON NameOfCatalog
    - This is not needed if you have a default catalog.
- You can also add WITH CHANGE_TRACKING =
    - MANUAL.
        - This means that the changes to the relevant data are tracked, but not automatically implemented into the full-text index.
        - The full-text index will only be updated when you run ALTER FULLTEXT INDEX NameOfIndex START UPDATE POPULATION.
        - You can also add this to the SQL Server Agent, if available, to run on a schedule.
    - AUTO.
        - Any changes to the data are automatically reflected in the full-text index. This is the default.
    - OFF
        - Changes to the relevant data are not tracked. The full-text index will only be populated if you run ALTER FULLTEXT INDEX NameOfIndex START FULL/INCREMENTAL POPULATION.
    - OFF, NO POPULATION
        - In addition to OFF, the full-text index will not be populated at that time – you will need to populate it later for the full-text index to be used, by running ALTER FULLTEXT INDEX NameOfIndex SET CHANGE_TRACKING MANUAL/AUTO/OFF;
- You can also populate it based on a timestamp (data type) column.
    - The first time that it is run, or if there is no timestamp column, then there will be a full population.
    - Afterwards, only rows added/deleted/modified will be updated in the full-text index.
    - To create/change a schedule in SSMS:

    - Right-hand click on the relevant table, and go to Full-Text Index – Properties.
    - Go to the Schedules page, and can go to New, Edit, or Delete an existing schedule.
    - Creating a New Schedule creates an SQL Server Agent job.

- You can check the indexes by using:
    - SELECT * FROM sys.fulltext_indexes;
    - Unlike other indexes, you can only create one full-text index per table.
- Once the full-text catalog and index have been set up, you can then write T-SQL queries without needing to reference them; SQL Server will do that itself.
- You can use:
    - CONTAINS – this matches words and phrases
        - Use in the WHERE or HAVING clause.
        - It returns TRUE or FALSE.
        - WHERE CONTAINS(ColumnName, SearchTerm)
            - ColumnName can be 1 column, multiple separated by commas, or * to search all full-text indexed columns.
        - To search for more than one term, you can use OR, AND and NOT within the term - 'Yes or No'.
        - To search for a term, enclose it in speech marks. For example: '"three star"'.
        - Use a * to look for a prefix – for example, 'beg*'. In phrases, the * is for applied to each word, and you only need 1 * – so 'beg cour*' can find “beginner course”.
        - You can look at variants of the word. For example, the following will find “geese” (note that FORMSOF is inside single quotation marks):
            - WHERE CONTAINS(FieldName, 'FORMSOF(INFLECTIONAL, "worked")')
        - For nouns, it can also find different forms of the word. For verbs, it can be find present/future/past tense, and different people.
        - To search for a word/phrase NEAR another word/phrase, you can use 'NEAR', and optionally say how close you want the words to be:

    - WHERE CONTAINS(FieldName, 'NEAR((Phillip, Burton), 3)') will find Phillip then Burton, or Burton then Phillip, within three terms.
    - You can use the * wildcard with NEAR (not '%').
    - If you need it to be in a particular order, then you can end with: ,3, TRUE)'). The order is the normal order of the language (left-to-right, but right-to-left for languages like Arabic and Hebrew).

- CONTAINSTABLE – this also matches words and phrases. However, this returns a table.
    - You should use it in the FROM clause or wherever else you use a table.
    - SELECT * FROM CONTAINSTABLE(TableName, ColumnName, SearchTerms).
        - The TableName can optionally contain the schema or the database name, separated by dots.
    - This will produce a table including a column called RANK. This shows how well the table is matched, from 0 to 1000.
        - You can use it to filter the results in the WHERE clause.
    - You can use a fourth term to limit the number of rows to be returned, based on the RANK – so: CONTAINSTABLE(TableName, ColumnName, SearchTerms,TopNumberOfRows)
    - There will also be a KEY column. This is the key column from the full-text index, and can be used in JOINs with the original table.
    - You can also use ISABOUT to created a weighted query, where there is more emphasis for certain words. The highest weight is 1.0.
        - 'ISABOUT (“FirstTerm”, SecondTerm WEIGHT(0.5), ThirdTerm Weight(0.2))'
- FREETEXT – this matches the meaning, not the exact word
    - WHERE FREETEXT(FieldName, SearchTerm)
- FREETEXTTABLE – this also matches the meaning

    - You cannot use WEIGHT, FORMSOF, wildcards or NEAR with FREETEXTTABLE.

61. Design for vector data, including vector data type, vector indexes, and size

- Vectors are arrays of numerical data which represent metadata (information about data), where most of the numbers are not zero.
- The VECTOR type generally stores numbers as float data. This is because:
    - It contains fractional numbers between -1 and +1.
    - The data does not need to be stored precisely, so float data allows for:
        - Approximation of data to be stored, with
        - A lower number of bytes needed for each number.
- You define the data type in a table column, such as VECTOR(1024).
    - This would define it as a float32 (single-precision), using 4 bytes per value.
    - The number in the brackets/paratheses shows how many dimensions are stored. It is up to 1,998.
    - It is generally either 2 times by itself multiple times (so, 512 or 1024), or 3 multiplied by 2 multiple times (for example, 768 or 1536).
- In SQL Server 2025, you can switch preview features on using
    - ALTER DATABASE SCOPED CONFIGURATION
    - SET PREVIEW_FEATURES = ON;
    - GO
- And then use the Preview float16, which uses only 2 bytes per value – for example VECTOR(1536, float16).

62a. Identify when to use vector-related types and functions for semantic searching, including VECTOR_NORMALIZE

- Different models can produce different unit lengths.
    - This does not apply to Azure OpenAI's models, which produce normalized vectors.

    - You should check whatever model you are using to see whether the result is normalised (of unit length) or not.

- If they are not, then you can use VECTOR_NORMALIZE to normalize a vector.
- It takes two arguments:
    - Firstly, the vector,
    - Secondly, the type of normalization:
        - norm1 – the sum of the absolute values (the value without a + or – sign) of the vector components,
        - norm2 – the Euclidean Norm, the square root of the sum of the squares of the vector components,
        - norminf – the infinity norm, the maximum of the absolute values of vector components.
- The result is a vector, which has:
    - The same direction as the input vector, but
    - A length of 1 according to the given norm.
- If the input vector was NULL, then the result would be NULL.
- You can also use VECTOR_NORM to return the vector's norm (its length or magnitude).

62b. Identify when to use vector-related types and functions for semantic searching, including VECTOR_DISTANCE

- Use the VECTOR_DISTANCE function when you want to use ENN for a vector search – see topic 63.
- The VECTOR_DISTANCE function calculates the distance between two vectors using a particular metric:
    - 'cosine',
    - 'euclidean', or
    - 'dot'.
- For example, VECTOR_DISTANCE('cosine', Vector1, Vector2)
- It returns a float which is the distance between the two vectors.
- You could then use ORDER BY and TOP to retrieve the best X results.

- Alternatively, you can use the WHERE clause to restrict it to a maximum of a specific distance.

62c. Identify when to use vector-related types and functions for semantic searching, including VECTORPROPERTY

- VECTORPROPERTY (with no underline) can be used to return the properties of a VECTOR.
- It is used as: VECTORPROPERTY(vector, property).
- There are two properties:
    - 'Dimensions' – the number of dimensions in the vector, and
    - 'BaseType' – the data type's name.

62d. Identify when to use vector-related types and functions for semantic searching, including VECTOR_SEARCH

- Use VECTOR_SEARCH when you want to do an ANN (Approximate Nearest Neighbors) search – see topic 63.
    - It uses a vector index, which requires at least 100 rows (see topic 64).
- It requires the PREVIEW_FEATURES to be enabled:
    - ALTER DATABASE SCOPED CONFIGURATION
      SET PREVIEW_FEATURES = ON;
      GO
- It takes 4 arguments:
    - TABLE
    - COLUMN (the Vector Column),
    - SIMILAR_TO (the Vector you are looking for)
    - METRIC (either 'cosine', 'dot' or 'euclidean'.
- For example:
    - VECTOR_SEARCH(TABLE = TableName, COLUMN = VectorColumn, SIMILAR_TO = SearchVector, METRIC = 'cosine') AS TableAlias
- It returns the table that it was given, together with a column called 'distance'.
- You can then use the distance column for ORDER BY distance ASC
    - You cannot use ORDER BY distance DESC.

    - You cannot sort by other columns.
- You cannot use VECTOR_SEARCH in a view.
- You can also use in the SELECT clause: TOP(N) WITH APPROXIMATE, to return the top N rows using ANN.

63. Choose between using ANN and ENN for vector search

- There are two different ways to do a vector search:
    - ENN – an Exact Nearest Neighbor, also known as a kNN, a k-nearest Neighbor.
    - ANN – an Approximate Nearest Neighbor.
- ENN does a comprehensive calculation across all indexed vectors.
    - It enables the highest accuracy, but it consumes a lot of resources, as it could take longer.
- Use it for:
    - When you need the absolutely best results,
    - When you have small datasets, or
    - When you have a big dataset, and can filter it beforehand to a small dataset.
- ANN uses indexing to speed up responses, but may not return the completely best Neighbor. Use this when:
    - You need quicker responses,
    - You have larger datasets, and
    - You don't need the completely best response.
- To use kNN/ENN, use the VECTOR_DISTANCE function, which requires you to look at every single row.
- To use ANN, use the VECTOR_SEARCH function.

64. Evaluate vector index types and metrics

- Vector indexes can be created to speed up the calculation of nearest neighbors.
- To use vector indexes, you first of all need to switch preview features on:
    - ALTER DATABASE SCOPED CONFIGURATION
    - SET PREVIEW_FEATURES = ON;

    - GO
- You can then create an index:
    - CREATE VECTOR INDEX NameOfIndex ON TableName (VectorColumn)
    - The VectorColumn must be of VECTOR type.
    - The table needs to have a primary key clustered index.
    - It needs at least 100 rows with non-NULL vector values.
- You can also add a METRIC:
    - WITH (METRIC = 'cosine')
        - Cosine similarity measures the cosine between two vectors in multidimensional space.
        - It is used in Azure OpenAI embedding models.
    - WITH (METRIC = 'euclidean')
        - Euclidean distance , or l2 norm, calculates the straight-line distance between two points.
    - WITH (METRIC = 'dot')
        - Dot product is the multiplication of two numbers, combining the magnitude (size) and cosine.
- To recreate it, you would need to DROP and CREATE it – you cannot ALTER it.
    - You would do this if you are updating the embedding, perhaps with a new model. You DROP the index, update the embedding, then CREATE it again.
- It is not recommended that you create vector indexes if you have a high percentage of duplicate vectors (duplication):
    - This would over-emphasis the duplicate rows, leading to poorer results.
- Finally, you can also add , TYPE = 'DiskANN' in the brackets/parentheses, although that is currently the only type you can use, so it is the default.

65. Implement vector search

- To implement vector search, you need to first of all create an Azure OpenAI resource in Azure:
    - Go to the Azure Portal (portal.azure.com),

    - Search for and create a new Azure OpenAI resource (go to Create - Azure OpenAI),
    - In the Basics tab, enter:
        - The subscription,
        - Resource Group,
        - Region (close to where the embeddings are going to be created),
        - Name, and
        - Pricing Tier – for Azure OpenAI, this will be the Standard tier.
    - In the Network tab, select what networks can use this resource.
        - If it is “Disabled”, then you need to add a Private Endpoint.
- Then you can go to "Review + Create" the Azure OpenAI resource.
- Once it has been deployed, in the Azure OpenAI resource:
    - Go to Resource Management - Keys and Endpoint, and copy the Endpoint.
    - click on "Explore Foundry portal"
        - Microsoft Foundry is the host of the AI model which generates the embeddings.
        - It does not store the data itself.
    - go to Deployments – Deploy model – Deploy base model - for example, the text-embedding-3-small.
    - Click on Confirm.
    - You can change the Deployment type (Global Standard, Data Zone Standard, and Standard).
    - Then click on "Create resource and deploy".
- You will need:
    - The OpenAI endpoint,
    - The Deployment Name, and
    - The API key.
- You will then need:

    - A table with the text and a VECTOR of the appropriate type – for the text-embedding-3-small, use VECTOR(1536).
    - You can Google the name of the base model, and "dimension size".
- Then create a database master key.
    - You will only need one per database.
    - CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'NameOfStrongPassword1!';
- Create a Database Scoped Credential:
    - CREATE DATABASE SCOPED CREDENTIAL [OpenAIEndpoint] WITH IDENTITY = 'Managed Identity', SECRET = '{"resourceid":"https://cognitiveservices.azure.com"}';
- Then reference that model in SQL Server.

```
CREATE EXTERNAL MODEL my_embedding_model
WITH (
  LOCATION = TargetURI',
  API_FORMAT = 'Azure OpenAI',
  MODEL_TYPE = EMBEDDINGS,
  MODEL = 'text-embedding-3-small',
  CREDENTIAL = [OpenAIEndpoint]
);
```

    - The Location should be the Endpoint for the Azure OpenAI, not the Foundry Model.
    - The Credential should be the name of the Database Scoped Credential, in hard brackets.
- Then you can use AI_GENERATE_EMBEDDINGS to update a vector column in a table with the embeddings.
    - UPDATE myTable
    - SET Embed = AI_GENERATE_EMBEDDINGS(myTable.text USE MODEL NameOfModel)
    - FROM myTable;

- An alternative way is by using the sp_invoke_external_rest_endpoint stored procedure - see topic 70.
- See topic 62.

66. Implement hybrid search

- Hybrid search is useful if you are not sure at the search time which is the best search – full-text or vector search.
- You create two queries, for each search, and write the results to a temporary table (which starts with a #).
- The advantages of hybrid search are:
    - It runs in SQL, thereby not needing additional components,
    - It is easy to create and can scale, and
    - Because it searches full-text and vector, the results can be more relevant.
- You then combine them together using a FULL OUTER JOIN, based on the ID column.

67. Implement reciprocal rank fusion (RRF)

- Reciprocal rank fusion (RRF) is the addition of the results from:
    - The Full-Text Search, and
    - The Vector Search.
- The method of calculating it is:
    - 1 divided by (rank plus 60).
    - 60 is a constant that is often used, but it can be changed.
- You then add the calculation from the Full-Text Search to the calculation from the Vector Search to create the RRF.
    - You should use COALESCE(result, 0.0) in case the result is NULL (it gets converted into zero, and then can be added to the other calculation).

68. Evaluate performance of vector and hybrid search

- To evaluate the performance of each of the different types of searches:
    - You run them,
    - You look at how long it took and resources it used,
    - You look at the results, and

- See which one gives you the results that is needed.

Design and implement retrieval-augmented generation (RAG)

69. Identify use cases for RAG

- RAG (Retrieval Augmented Generation) uses AI which is grounded (based) or enriched on your data.
    - Without your data, it will be based on whatever it was used to train it.
- For example:
    - You might have a database of a conference, and want to query it in natural language.
    - You have data that you want to use to make decisions, summarize data and create presentations.
    - You want to solve problems based on your troubleshooting information and organization policies.
    - Answer questions based on a customer's purchases, or your latest sales.
- The advantages over training a custom model include:
    - Time taken – once the RAG is set up, it does not need additional time,
    - Updated data – if the data changes, the RAG will look at the latest data; a custom model would need to be retrained..
- The overall process is:
    - Retrieve relevant data from the table,
    - Create a prompt which includes that data as context,
    - Send it to the LLM,
    - Retrieve and convert the response.

70. Create a prompt by using the sp_invoke_external_rest_endpoint stored procedure

- In SQL Server 2025, this stored procedure needed to be enabled using sp_configure.

-- Allow advanced options


```
EXEC sp_configure 'show advanced options', 1;

RECONFIGURE;

-- Enable external REST endpoint calls

EXEC sp_configure 'external rest endpoint enabled', 1;

RECONFIGURE;
```

- You can use this stored procedure to call Azure SQL Database:

```
EXEC @returnvalue = sp_invoke_external_rest_endpoint

@url =
N'https://<endpoint>.openai.azure.com/openai/deployments/<model>/chat/
completions?api-version=<api-version>',

@method = 'POST',

@payload = @payload,

@credential = [https://<endpoint>.openai.azure.com],

@retry_count = 5,

@response = @response OUTPUT;
```

- The @url is the Azure OpenAI deployment endpoint.
- For generation requests, you usually use the POST method.
- The @payload is the JSON prompt.
- The @credential is the database scoped credential.
- The @retry_count shows the number of times it will call this procedure.
- The @response will contain the response. It uses OUTPUT to store the value.
- Go to topic 73.

71. Convert structured data to JSON for language model processing

- To convert your structured data to JSON, you add to the end of the query:
    - FOR JSON PATH; (the semicolon is optional)
- You can also add
    - , WITHOUT_ARRAY_WRAPPER. This suppress [ ] square hard brackets if you have just one record/row.

- , INCLUDE_NULL_VALUES would keep NULL fields/columns. If you don't have it, then any NULL values would be suppressed.
- , ROOT('root_name') puts the output into a root element.

72. Send results to language model

- First of all, you should create a ChatGPT deployment:
  - Go to the new Microsoft Foundry (ai.azure.com),
  - Go to Build - Models, and click on "Deploy a base model".
  - Search for "gpt", select a model, then click on Deploy - Default Settings.
- After that, you need to create a chat structure.
- In the 'messages' or 'input' node, you need to define two roles:
  - 'role': 'system' should have the “content” which defines what the model should do.
    - For example, 'content': ' You are an AI assistant that helps people find information about reviews.'
  - 'role': 'user' should have the question and any context.
    - For example: 'content': 'The Context is: ...' + CHAR(10) + CHAR(10) + 'The Question is ...'
  - You can use NVARCHAR(MAX) to store these in, using JSON_OBJECT and JSON_ARRAY – for example:
    - JSON_OBJECT('messages': JSON_ARRAY(
    - JSON_OBJECT('role': 'system', 'content', @strSystem)
    - JSON_OBJECT('role': 'user', 'content', @strUser)), ...)
  - You can also add additional parameters in the ... above, including:
    - 'max_tokens': ###. This is the maximum number of tokens that will be used in the response (you pay for each token)
    - 'Temperature': ###. This is a number between 0 and 2.
      - Lower numbers will create more factual answers.
      - Higher numbers will create more creative and less deterministic answers.

- You can create an embedding for the question, using either AI_GENERATE_EMBEDDINGS or sp_invoke_external_rest_endpoint
    - SELECT @vecQuestion = AI_GENERATE_EMBEDDINGS(@strQuestion USE MODEL model_name)
- Continued in topic 70.

73. Extract language model responses

- If there is a valid response, then @returnvalue will be zero.
- The @response will contain:
    - Response – status – http, which contains the code (for example, 200) and description (of the code).
    - Result – choices – message, which contains role (which will be “assistant”) and content (which is the response you are looking for).
- If @returnvalue is zero, then you can retrieve the content using JSON_VALUE:
    - SET @answer = JSON_VALUE(@response, '$.result.choices[0].message.content');
- If it is not zero, then you might want to retrieve the description:
    - JSON_VALUE(@response, '$.response.status.http.description')

